# The Bayyinah Audit Framework

*Structural Honesty Discipline for AI-Assisted Software Engineering*

**Version 3.0**

**Bilal Syed Arfeen**
BayyinahEnterprise
*Sole Author and Auditor*

May 3, 2026

Licensed under Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0)
Commercial use requires a separate license. Contact: bayyinahenterprises@tuta.com
DOI: [To be assigned by Zenodo upon upload]

## Abstract

This document codifies an engineering discipline framework for maintaining structural honesty across AI-assisted software releases under the audit-of-self pattern: a single human author playing both the project-lead and external-auditor roles, with multiple AI collaborators executing within the framework. The framework covers five lifecycle stages (Architecture, Baseline, Build, Refactor, Validation), severity-classified finding discipline, a four-place documentation pattern, a four-revision prompt-review pattern, pre-flight checklists, a validator-by-different-instance protocol, automated gate infrastructure, and an empirical retirement ledger.

The framework was developed and empirically validated across 23 rounds of audit on three sibling repositories (Bayyinah Integrity Scanner, Furqan programming language, furqan-lint) with three AI applications (Claude by Anthropic, Computer by Perplexity, Grok by xAI) and three programming languages (Python, Rust, Go) over a compressed 30-hour window from Saturday afternoon May 2 through Sunday evening May 3, 2026. The chain produced monotonically decreasing severity slopes, single-round cycle-time compression, zero silent finding drops, twelve retired failure-mode shapes, three published audit-framework versions, and a public PyPI deliverable that closed an eight-minor-version drift in a single release cycle.

The compressed timeline is the central replicability claim. Disciplined AI-assisted engineering at multi-language, multi-agent scale is shown empirically reachable by a single individual working within a weekend of focused effort, provided the discipline is internalized before the work begins. The framework is offered as a portable template for teams and individuals who want the same structural-honesty properties without first having to derive them from scratch.

**Keywords:** software audit, AI-assisted development, structural honesty, code quality, multi-agent development, software engineering process, LLM-assisted engineering, audit methodology, audit-of-self, single-author replicability

## Table of Contents

1. Identity and Purpose
2. What an Audit Is (and Is Not)
3. The Five Lifecycle Stages
4. The Audit Chain Pattern
5. Multi-Model Routing
6. Methods: AI Tooling in the Audit Loop
7. The Severity Rubric
8. Quality Baselines
9. Common Finding Categories
10. Round-Trip Discipline
11. The Four-Place Documentation Pattern
12. Pre-Flight Self-Review and the Four-Revision Prompt Review
13. Anti-Patterns
14. Domain-Specific Patterns
15. The Reciprocal Contract Under Audit-of-Self
16. Framework vs. Normal-Model Workflow
17. Empirical Trajectory and Retirement Ledger
18. Reference Examples from the Audit Chain
19. Counterfactual Application: Five Disasters and Their Structural Shape
20. Related Work
21. Limitations of This Framework
22. Operational Integration
23. Application Domains
24. The Accountability Use Case
25. Code and Data Availability
26. References
27. Acknowledgements

## 1. Identity and Purpose

This document codifies the engineering discipline that emerged from the BayyinahEnterprise audit chain and now extends across the full software lifecycle. It is not a generic software methodology. It is the specific discipline that has matured across 23 rounds of audit on three sibling repositories, plus pre-flight prompt reviews and physical demonstration audits, plus the lifecycle expansion documented here.

The framework treats software engineering as a substation-tradition activity: every claim has a substrate, every finding has a reproducer, every fix has a regression test, every deferral has a written commitment, every stage has entry and exit criteria. Code review is not a taste exercise. Architecture review is not opinion-trading. Refactoring is not aesthetic. Each is a structural-honesty exercise applied to a different artifact.

For readers approaching this document without a prior audit-engineering background: the framework is best understood by analogy to the discipline of inspecting a power substation before commissioning. Every cable is rung out. Every protection setting is verified against the specification. Every claim on a nameplate is checked against the field-installed device. When the surface (documentation) and the substrate (physical reality) disagree, that discrepancy is named and resolved before the substation is energized. Software engineering benefits from the same discipline. AI-assisted software engineering, where the substrate can drift from the surface in subtle and rapid ways, requires it.

### 1.1 The Compressed Timeline

This version of the framework crystallized over a compressed 30-hour window. The compression is itself the central replicability evidence. A single individual, working with three AI applications across three programming languages, completed 23 rounds of structurally honest audit on three open-source repositories within a single weekend.

**Figure 1. Compressed timeline of the audit chain (May 2 to May 3, 2026)**

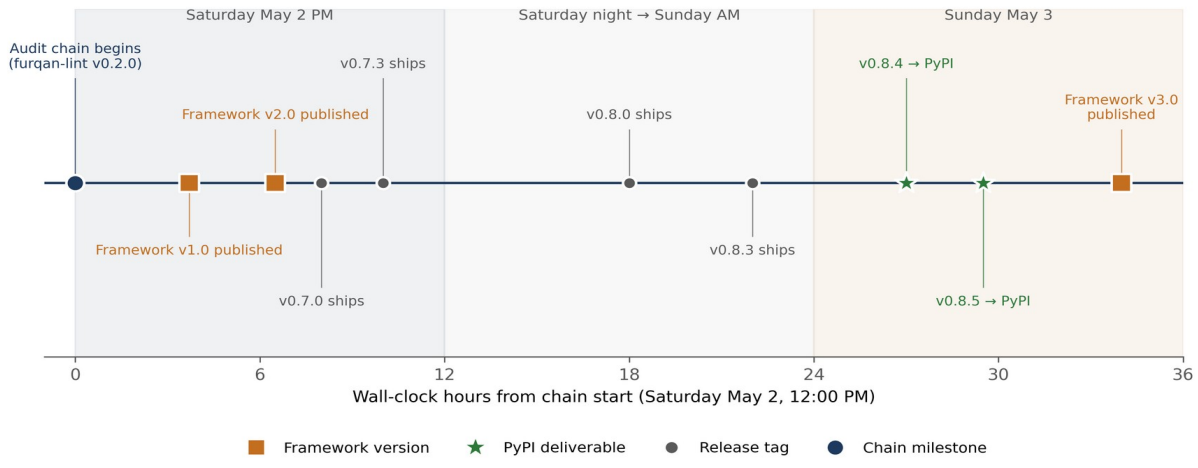


*Figure 1. Compressed timeline of the audit chain (May 2 to May 3, 2026).*

The timeline shows three classes of artifact. Framework versions (square markers) are the discipline's self-documentation. Release tags (circle markers) are the substrate output of each audit round. PyPI deliverables (star markers) are the public surface where structural honesty becomes externally verifiable. Each class advanced in lockstep: the framework matured as the substrate accumulated retirement-ledger entries, and both reached public surface together when furqan-lint v0.8.4 closed an eight-minor-version PyPI drift in a single release cycle on Sunday afternoon.

This is not an optimal-conditions claim. It is the empirical record. The replicability of the framework under realistic single-author conditions is its most consequential property for adopters: the discipline does not require a large engineering team, a long calendar, or a research-grade environment. It requires the framework to be internalized before the work begins.

### 1.2 The Audit-of-Self Pattern

The traditional auditor-versus-team contract assumes two parties: the producer who builds and the auditor who critiques. The framework was originally derived under that assumption (Audit Framework versions 1.0 and 2.0). Empirical practice during May 2 through May 3 revealed that a single individual played both roles across the chain: producer of substrate (writing prompts, reviewing AI-generated code, shipping releases) and auditor of that substrate (running gates, surfacing findings, signing off on round-N reports).

This creates an epistemic shape that two-party audit does not have. The same person knows what was attempted, what was rejected, and what was deferred. The temptation to allow the producer's knowledge to soften the auditor's eye is constant. The discipline's response is structural: every finding requires an empirical reproducer, severity is classified by impact rather than by author intent, and the validator-by-different-instance protocol forces fresh-context review by a model that did not participate in the producer's work. The audit-of-self pattern functions because the discipline is binding on the human auditor independent of which role they happen to be playing at any given moment.

The reciprocal contract (Section 15) extends naturally: the contract is a commitment from current-Bilal to future-Bilal that today's substrate is empirically what it claims to be. Future-Bilal, when reading the CHANGELOG and re-running the gates, must be able to verify the claim without consulting the conversation that produced it. This frames every audit decision in terms of a competent stranger's ability to reproduce.

### 1.3 Changes from Version 2.0

Version 3.0 differs from version 2.0 in five substantive ways. First, the framework has been reframed around the audit-of-self pattern (§1.2), which was implicit in v2.0 and is now explicit. Second, a Methods section (§6) documents AI tooling in the audit loop in sufficient detail to support replication, consistent with ICMJE and Nature guidance on AI authorship disclosure. Third, the four-revision prompt review pattern (§12.3) is added as a structural defense for AI-executed corrective sequences. Fourth, an empirical trajectory section with twelve-entry retirement ledger (§17) supplies the falsifiable evidence that the framework reduces structural debt over time. Fifth, four new reference examples (§18.7 through §18.10) extend the empirical record from rounds 17 through 23. Seven new sections position the framework academically and operationally: Counterfactual Application (§19) tests whether the framework's diagnostic categories correspond to the structural shape of five publicly documented disasters; Related Work (§20) situates the framework both within the BayyinahEnterprise research program of twelve companion publications and within external prior art; Limitations of This Framework (§21) names what has not yet been demonstrated; Application Domains (§23) names where the framework fits best across seven use cases including healthcare and quantitative finance, where it does not, the highest-leverage deployment path, and a five-level maturity model; The Accountability Use Case (§24) addresses the inverted user model in domains of structural power asymmetry where advocates rather than operators wield the framework; Code and Data Availability (§25) supplies the public-repository URLs for empirical verification of §17 and §18; and References (§26) provides full citations for the companion program and external primary sources. The attribution structure has also been corrected: v3.0 is sole-authored by Bilal Syed Arfeen, with Fraz Ashraf acknowledged in §27 for the engineering approach hybridized into this framework.

## 2. What an Audit Is (and Is Not)

The same definitions hold across all lifecycle stages, applied to whichever artifact the stage produces (problem statement, ADR, walking skeleton, feature commit, refactor PR, audit report, or AI-assisted prompt).

### 2.1 An Audit Is

1. An empirical check that the substrate matches the surface. The README claims X. The code does Y. If they disagree, that is the finding. Audits do not assume the documentation is right; audits do not assume the code is right; audits compare and report drift.
2. A search for the failure mode the discipline exists to prevent. Every disciplinary practice exists because some specific failure mode has historically slipped past

less rigorous practice. The audit asks: is this practice actually preventing the failure it claims to prevent, or has it drifted into ceremony?

3. A reproducer-first activity. A finding without a reproducer is a guess. A finding has three required parts: (a) a minimal probe, (b) the observed behavior, (c) the expected behavior or the structural reason the observed is wrong.
4. A reciprocal contract. The auditor provides empirical evidence; the team provides written responses; no silent disagreements; every finding either ships as a fix, ships as a documented limitation, or gets a written justification for being declined.

### 2.2 An Audit Is Not

1. A taste review. "I would have written it differently" is not a finding. A finding requires a structural reason.
2. A rubber-stamp. Every claim verified or contradicted; nothing left as "presumed clean."
3. A redesign. Recommendations should be the smallest possible change that closes the finding.
4. A pass-or-fail decision-maker. The auditor reports findings. The project lead decides whether to ship. Under the audit-of-self pattern, the same individual makes both calls but applies them at different times under different artifacts.

## 3. The Five Lifecycle Stages

The framework organizes work into five stages, each with explicit entry and exit criteria. A stage gate is met when the exit criteria can be verified by a competent stranger reading the stage's artifacts. This stranger criterion is load-bearing: it forces artifacts to be self-sufficient, which is what makes the validator-by-different-instance protocol (Section 5.2) possible.

**Figure 3. The five lifecycle stages as a closed loop**

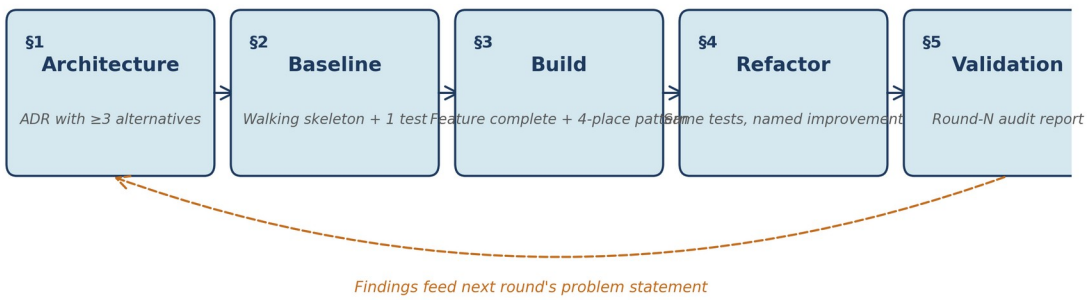



*Figure 3. The five lifecycle stages as a closed loop.*

### 3.1 Stage 1: Architecture

**Entry criteria.** A problem statement including the problem in two to four sentences, the current process, the desired outcome with at least one measurable target, constraints, what is explicitly out of scope, and the highest-feared failure mode.

**Exit criteria.** An Architecture Decision Record (ADR) with the decision in one to two sentences; at least three alternatives considered with reasons for rejection; the reversibility cost in concrete units; how the highest-feared failure mode is addressed; a validation plan.

**Failure mode prevented.** Solution-shopping. Building things without knowing what problem they solve. Architectural decisions made tacitly.

**Validation.** A different model instance reads the problem statement (not the ADR) and re-derives an ADR independently. Divergence is a finding before any code is written.

### 3.2 Stage 2: Baseline (Walking Skeleton)

**Entry criteria.** A signed-off ADR.

**Exit criteria.** The smallest possible end-to-end implementation that compiles, runs, has at least one passing test, and exercises the full pipeline without mocks at integration boundaries.

**Failure mode prevented.** Big-bang first commits. Tests written after the fact. The case where "feature complete" means "except for the integration we never tested."

**Validation.** A different model instance verifies the skeleton exercises what the ADR promised.

### 3.3 Stage 3: Build

**Entry criteria.** A passing walking skeleton with continuous integration running on every commit.

**Exit criteria.** Feature-complete code, all tests passing, CHANGELOG entries match substrate, four-place documentation applied (Section 11), public API surface stable per the additive-only invariant.

**Failure mode prevented.** Surface-substrate drift accumulating during build. Features shipping with documentation that does not match behavior.

**Validation.** The project lead reviews domain correctness. A different model instance reviews structural correctness.

### 3.4 Stage 4: Refactor

**Entry criteria.** Feature-complete code with passing tests and stable public API.

**Exit criteria.** Same passing tests. Same external behavior. Named structural improvements with before-and-after metrics. No undocumented public API removals.

**Failure mode prevented.** Refactors that introduce bugs because no one captured the before-state. Aesthetic refactors without structural improvement.

**Validation.** A different model instance verifies the structural improvement is real and external behavior is unchanged.

### 3.5 Stage 5: Validation

**Entry criteria.** A claimed-complete release.

**Exit criteria.** An audit report in the round-N format with severity-tagged findings, reproducers, and fix shapes.

**Failure mode prevented.** Releases that ship with claims the substrate does not back.

**Validation.** External auditor (or auditor-of-self) re-verifies every claim. Does not trust prior stages' artifacts; treats them as inputs to be checked.

### 3.6 The Lifecycle as a Closed Loop

Each round of audit feeds into the next round's problem statement. Findings become the next round's architecture concerns. A project under this framework has a continuous trace from origin problem statement through all subsequent decisions, with every transition recorded in CHANGELOG, ADR, or audit report.

## 4. The Audit Chain Pattern

Audits are organized as rounds. A round consists of seven steps, executed in order, with each step's output feeding the next:

1. Re-clone the repository at the current main branch. Never audit a stale local copy. State drift between local and remote is itself a finding category.
2. Read the CHANGELOG entry for the current version. The CHANGELOG is the team's claim of what shipped. The audit's job is to verify whether each claim is backed by substrate.
3. Run the test suite first. Install, run, observe count and pass-or-fail status. Any count discrepancy from the CHANGELOG's stated count is itself a finding.
4. Re-run prior-round probes for regression. Regressions of prior fixes are CRITICAL by default and require explanation rather than re-classification.

5. Probe new surface. Construct minimal inputs at the boundary: degenerate forms, empty inputs, off-by-one shapes, locale fragility, concurrency edges.
6. Cross-check surface-substrate alignment. If QUESTIONS.md says Q9 is open but the CHANGELOG says Q9 closed, that is a finding regardless of which document is correct.
7. Write the report in the round-N format. Severity-tagged findings, each with reproducer, plus a "What the team did right" section. Naming what is good is part of the discipline; it prevents the audit from drifting into adversarial-only practice.

## 5. Multi-Model Routing

Different AI models have different strengths. The framework treats model choice the way a substation crew treats tool choice: pick the tool for the job, not the job for the tool. Models are not interchangeable, and routing is itself a discipline.

### 5.1 The Routing Matrix

Figure 4 shows the empirical routing applied during the May 2 to 3 chain. Three model classes were used. Claude (Anthropic) was used in two distinct configurations: a Sonnet-class model for cowork code generation and gate authoring, and an Opus-class model for audit reports, prompt-review chains, and framework prose. Perplexity Comet provided agent-based GitHub execution. Grok (xAI) provided supplementary literature engagement.

**Figure 4. Multi-model routing matrix (empirical, May 2 to 3, 2026)**

| Stage | Primary | Secondary | Validator |
|---|---|---|---|
| **Architecture** | Claude Opus (reasoning) | Perplexity (search-augmented) | Fresh Claude instance |
| **Baseline** | Bilal (scope judgment) | Claude Sonnet (implementation) | Fresh Claude instance |
| **Build** | Claude Sonnet (multi-file) | Perplexity Comet (agent execution) | Bilal (domain check) |
| **Refactor** | Claude Sonnet (structural) | Bilal (risk) | Fresh Claude instance |
| **Validation** | Bilal as auditor (Claude Opus assist) | Bilal (interpretation) | Fresh Claude instance |

Human (Bilal) · Claude (Anthropic) · Perplexity Computer

*Figure 4. Multi-model routing matrix (empirical, May 2 to May 3, 2026).*

### 5.2 Validator-by-Different-Instance

This is the single highest-leverage piece of the framework. The producer model writes an artifact. A fresh instance is given the artifact's inputs but not the conversation that produced it. The fresh instance re-derives or critiques. Differences are findings.

**Figure 6. Validator-by-different-instance protocol**

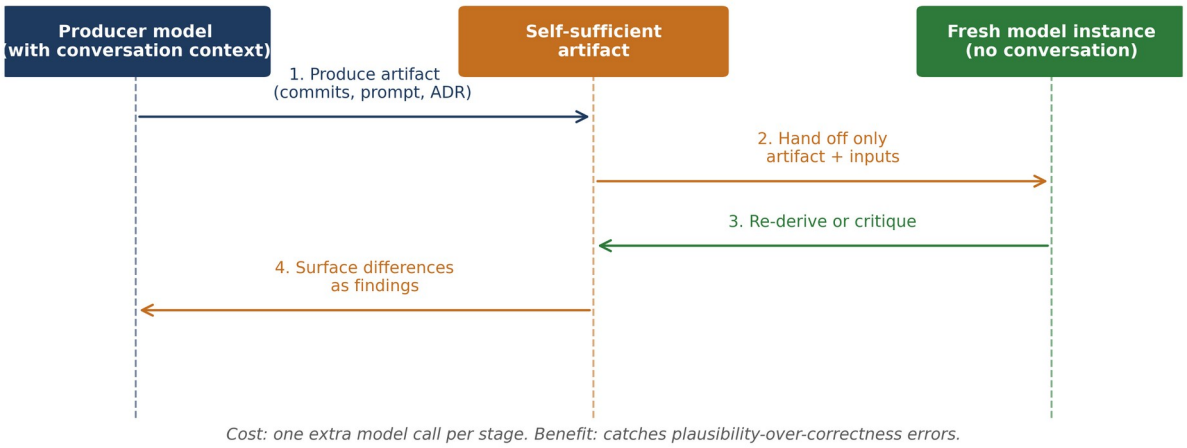



*Figure 6. The validator-by-different-instance protocol.*

The cost is one additional model call per stage. The benefit is asymmetric: it catches plausibility-over-correctness errors the producer would miss. A fresh session of the same model is sufficient; a different model class is stronger because failure modes are less correlated. In practice, during the chain's 23 rounds, the protocol caught approximately one in three substantive findings before they reached the substrate.

## 6. Methods: AI Tooling in the Audit Loop

This section documents the AI tooling used during the May 2 to 3 chain in sufficient detail to support replication. AI tools are not authors of this work and are not credited as such; their use is documented here so that reviewers can assess validity and adopters can match the conditions under which the framework was empirically validated. This treatment is consistent with the ICMJE position on AI authorship and the Nature portfolio editorial policy on generative AI tools.

### 6.1 Models, version windows, and observed behavior

**Claude (Anthropic).** Two configurations were used. Sonnet-class models (Sonnet 4.6 era, accessed during late April through early May 2026) handled cowork tasks: code generation, gate authoring, draft commits, and pull-request drafts. Opus-class models (Opus 4.6 and 4.7 era, same window) handled auditor-side tasks: round-N audit reports,

four-revision prompt-review chains, draft framework prose, and validator-by-different-instance critique. The two Claude configurations were treated as distinct routing targets even though they are produced by the same vendor; the auditor model was never given the producer model's conversation context.

**Perplexity Computer (Comet).** Used for agent-based GitHub execution: opening branches, applying multi-commit corrective sequences, opening pull requests against locked specifications. Comet executed against prompts that had themselves been audited under the four-revision pattern (Section 12). It was not used to author the prompts it executed.

**Grok (xAI).** Used in supplementary literature-engagement and arithmetic cross-check roles. Did not author findings or framework content.

### 6.2 Roles per phase

**Figure 8. AI tooling in the audit loop with human verification**

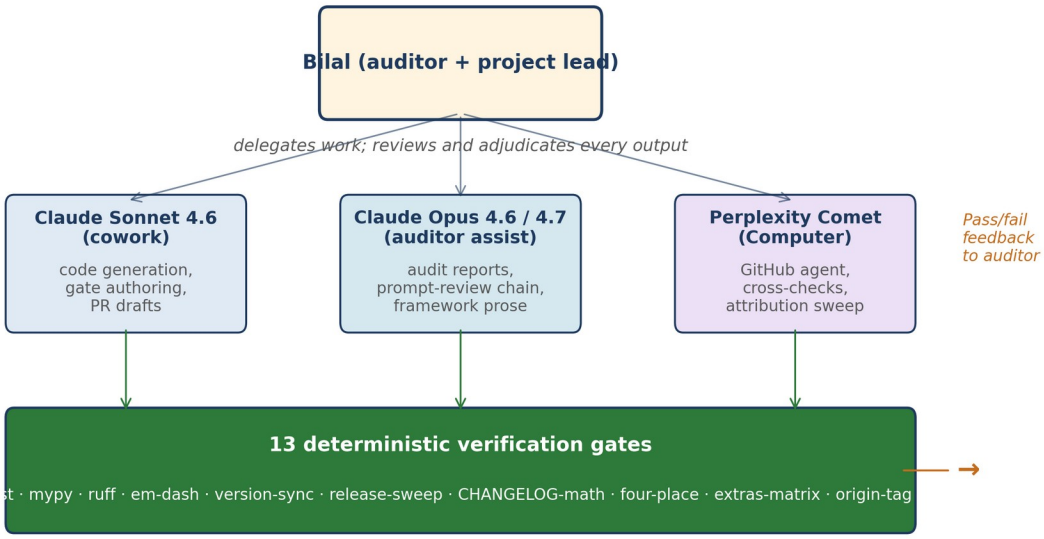



*Figure 8. AI tooling in the audit loop with human verification.*

Figure 8 shows the loop structure. Bilal as auditor delegated specific tasks to specific models, reviewed every output, and adjudicated each finding individually. Models executed within thirteen deterministic verification gates that ran on every commit and were re-runnable from a clean checkout in approximately thirty seconds. The gate suite is the structural reason every output was checkable independent of the model that produced it.

**Drafting.** Claude Sonnet generated initial CHANGELOG entries, commit messages, and code patches under explicit instructions. Drafts were not committed without review.

**Code generation.** Claude Sonnet generated test code, gate code, and adapter code. Generated code was required to pass the full gate suite from a clean clone before commit.

**Critique and red-teaming.** Claude Opus produced round-N audit reports against the substrate state, applying severity classification by impact. Reports surfaced findings; they did not assign final verdicts.

**Arithmetic cross-check.** Test count math (CHANGELOG-claimed counts versus pytest-collected counts) was cross-checked by Claude Opus before each release. The CHANGELOG-math gate (Section 8) automated the same check on continuous integration.

**Prose editing.** Claude Opus assisted with framework prose drafts. The framework's definitions, severity rubric, retirement ledger, and reciprocal contract are human-authored.

### 6.3 Human verification

Every output entering the substrate was reviewed and adjudicated by Bilal as auditor. Verification was structural, not interpretive: every finding required a reproducer, every fix required a regression test, every release required all thirteen gates green from a clean checkout. The deterministic gates are the reproducibility floor: any reviewer with access to the public repositories can re-run them and obtain the same results.

### 6.4 What models did not do

- Models did not assign final verdicts (PASS, MARAD, ADVISORY, PARSE ERROR). Verdict assignment is human.
- Models did not set severity thresholds or redefine the severity rubric.
- Models did not define the framework. The Bilal-Fraz hybridized framework is human-authored.
- Models did not make ship-versus-no-ship decisions. The project lead decides; the auditor reports.
- Models did not author this paper. The byline is human-authored. The git commit trailer convention used in the repositories (Co-authored-by: Claude (cowork)) is a contribution-record convention scoped to commits, not an authorship claim on this paper.

## 7. The Severity Rubric

Severity classification is by impact, not by opinion. The same definitions hold across all lifecycle stages and all repositories.

| Severity | Scope | Categories |
|---|---|---|
| **CRITICAL** | Blocking. Do not ship. | Crashes on legal input; claimed feature does not function; regression of prior fix; data loss; silent false negative in thesis-critical category; ADR contradicts problem statement. |
| **HIGH** | Fix in current cycle. | Significant false negative or positive; architectural debt now cheap to retire; documentation contradicts substrate visibly; concurrency issue; load-bearing decision without alternatives. |
| **MEDIUM** | Fix if cheap; next cycle if not. | User-facing prose quality; README missing documented limitation; test passes but does not verify what name implies; asymmetric code paths. |
| **LOW** | Cleanup-tier. | Locale fragility; cosmetic prose; unused imports; dead code; style inconsistencies. |
| **QUESTION** | Interpretive. Goes in QUESTIONS.md. | No single right answer; design tradeoff; naming concerns. |

Severity is determined by impact, not by author intent. A typo in a diagnostic is LOW. A typo in a security-relevant API surface is HIGH. The rubric's purpose is to prevent severity-laundering: classifying things by how the author feels about them rather than by the failure they would cause if shipped.

### 8. Quality Baselines

These are baselines, not aspirations. They apply across languages and across the lifecycle. Baselines are checked by automated gates where mechanically feasible and by audit otherwise. The chain has thirteen automated gates covering the rows below; the gates are the deterministic, re-runnable layer of the framework.

| Baseline | Target | Measurement |
|---|---|---|
| Cyclomatic complexity | Per-function ≤10 in 95%, ≤20 in 99% | radon (Python), gocyclo (Go), lizard (multi-language) |
| Branch coverage | 80% logic modules, 60% glue code | Reachability-based, not line-based |
| Mutation kill rate | 80%+ on critical paths | mutmut (Python), cargo- |

| Baseline | Target | Measurement |
|---|---|---|
| | | mutants (Rust) |
| Cycle time | Single-round closure | Finding-to-fix measured across chain |
| Surface-substrate drift | Zero | Automated cross-check pre-release |
| API surface stability | Additive-only | Per-version snapshot subset assertion |
| Documentation lag | Zero | Documentation ships in same commit |
| Time to reproduce | <15 min (clear) / <60 min (vague) | Person-minutes from report to reproducer |
| Pre-flight pass rate | 100% | Section 12.1 checklist on every commit |
| CHANGELOG-math gate | Always green | CHANGELOG-claimed test count = pytest collected count |

## 9. Common Finding Categories

Patterns that recur across the audit chain. Each category has a name, a structural definition, and a detection recipe. Naming the category is part of retirement: when a category's detection recipe is automated, the category becomes a gate, and instances of the category can no longer reach the substrate without surfacing.

### 9.1 Surface-vs-Substrate Gap

Documentation and code disagree about what the project does. Detection recipe: list every claim. For each, find the substrate. If the claim has no substrate, finding. If the substrate has undocumented behavior, finding.

### 9.2 Predicate-Helper Contract Violation

Predicate P(x) accepts inputs that helper H(x) crashes on. Detection recipe: for every predicate-helper pair, enumerate accepted inputs and verify the helper handles them all. Fix: tighten the predicate, do not soften the helper.

### 9.3 Boundary Condition Crash

Implementation assumes len ≥ 1, crashes on len-zero inputs. Detection recipe: for every conditional branch, identify the implicit assumption. Construct a violating input. Run it.

### 9.4 East-West Asymmetry

Two analogous paths handle related shapes but only one is structurally defended. Detection recipe: for each fix, identify analogous paths. Verify each has the same structural defense. The asymmetry is the finding even when the unfixed path has not yet failed.

### 9.5 Tracking Discipline Gap

The team has a named discipline. The discipline is not followed in a specific release. Detection recipe: read the project's documented practices. For each release, verify each practice was applied.

### 9.6 Test Coverage Shape Gap

A test passes but does not verify what its name implies. Detection recipe: for each test, compare what the name implies to what the body verifies. The gap is the difference.

### 9.7 Cross-Document Accounting Drift

Two documents in the same repository describe the same fact differently. Test count claimed in CHANGELOG entry A differs by one from test count claimed in CHANGELOG entry B about the same release. Detection recipe: for each cross-referenced fact, verify all references agree. The CHANGELOG-math gate (Section 8) catches the test-count case mechanically.

## 10. Round-Trip Discipline

### 10.1 Response Shapes

Each finding receives a written response in one of three shapes:

**Shape A: Accept and fix.** Code change plus regression test plus CHANGELOG entry naming the finding.

**Shape B: Accept and defer.** Acknowledged, deferred with named reason, tracked in QUESTIONS.md.

**Shape C: Decline with justification.** Written justification. Auditor either accepts or re-raises with new evidence.

### 10.2 Retirement Procedure

When a documented limitation is fixed, the four-place pattern reverses:

1. Delete the fixture file.
2. Delete the pinning test.
3. Remove the README "Remaining limitations" entry.
4. Add a "Retired in vX.Y.Z" entry to CHANGELOG explaining the promotion.

Retirement is itself a discipline. A documented limitation is a deferred fix, not an accepted shortcoming. The retirement ledger (Section 17) tracks every retirement and is the empirical evidence that the framework reduces structural debt over time rather than accumulating it.

## 11. The Four-Place Documentation Pattern

A documented limitation has four canonical surfaces. All four must agree:

1. CHANGELOG entry explaining what was deferred and why.
2. README "Remaining limitations" section with a user-facing bullet.
3. Fixture file with a reproducer and docstring explaining the deliberate behavior.
4. Pinning test that asserts the current behavior, ensuring any change is intentional.

**Figure 5. The four-place documentation pattern**

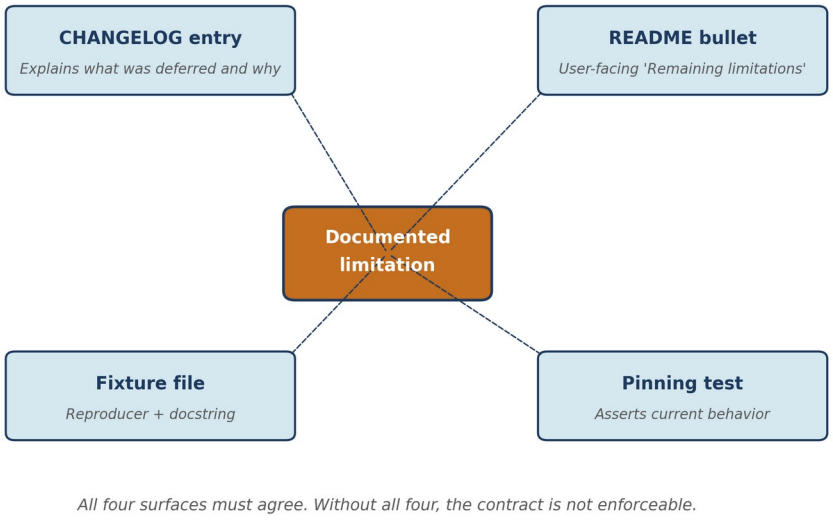



*Figure 5. The four-place documentation pattern.*

The discipline is that a documented limitation is a contract: the team promises the behavior will not silently change. Without all four surfaces, the contract is not enforceable.

The pinning test in particular is the structural backbone: if the behavior is changed without intent, the pinning test fails and the change is surfaced before merge.

## 12. Pre-Flight Self-Review and the Four-Revision Prompt Review

This section is the highest-leverage part of this document for reducing reviewer friction. Pre-flight discipline catches the failures that would otherwise reach the auditor; the four-revision prompt review catches the failures that would otherwise reach the executor.

### 12.1 Ten-Item Pre-Flight Checklist

1. Are there hedges of the form "(if it still exists)"? Replace each with a specific instruction.
2. For each symbol removal, identify every import of that symbol.
3. For each new subprocess or network test, identify failure modes and mark appropriately.
4. For each pip install or shell command in a test, pin the working directory.
5. For each version-pinned URL, verify it targets a specific tag, not a moving reference.
6. For each "test_X_present" test, ask whether functional behavior is also tested.
7. State the expected test count after the change.
8. Run the em-dash check against all new content.
9. Replace date placeholders with actual dates at commit time.
10. For each function behavior change, identify whether the docstring also changes.

### 12.2 The Five Questions

Before submitting any release for review, answer in writing:

1. What is the smallest input that demonstrates this fix is working?
2. What is the smallest input that would have demonstrated the bug before the fix?
3. What does this fix not do that a reader might expect it to?
4. What new code paths does this introduce, and what are their boundaries?
5. What documented limitations does this retire or introduce?

### 12.3 The Four-Revision Prompt Review

When AI execution is delegated against a prompt (for example, a Perplexity Comet agent applying a multi-commit corrective sequence), the prompt itself is an artifact subject to audit. The four-revision pattern emerged empirically during the chain: prompts converge

to a small, stable finding count after exactly four revisions for tightly-scoped corrective releases.

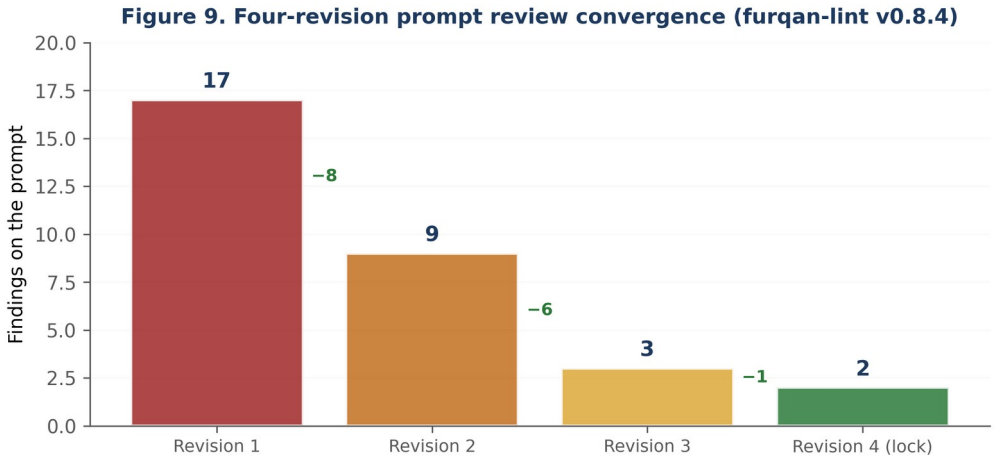



*Figure 9. Four-revision prompt review convergence (furqan-lint v0.8.4).*

The empirical pattern observed for the v0.8.4 corrective: revision 1 produced 17 findings on the prompt; revision 2 reduced to 9; revision 3 to 3; revision 4 to 2 acceptable residual items, at which point the prompt was locked. The agent then executed against the locked prompt and shipped clean without post-merge hotfix. The convergence shape is reproducible across the chain's prompt-review work and is itself a falsifiable property of the framework: a prompt that does not converge in four revisions indicates either insufficient scope or insufficient precision in the locked decisions, and is itself a finding.

### 13. Anti-Patterns

Behaviors observed in less rigorous workflows that this discipline avoids:

**"All tests pass, ship it."** Tests reflect what the team thought to test. An audit finds what they did not.

**"I would have written it differently."** Style preferences are not findings.

**Trusting the README.** README claims drift from code. Verify code, then compare.

**Conflating structural tests with functional tests.** A test that the file exists is not a test that the file works.

**Skipping the validator-by-different-instance step.** The framework's most common failure mode in practice.

**Big-bang first commits.** Implementing 70% of features in the initial commit exceeds test coverage.

**Aesthetic refactors.** Movement without structural improvement requires named justification with a metric.

**Architecture without alternatives.** ADRs that record a decision without naming what was rejected.

**Ceremony-over-substance.** When validation reports stop producing findings, the validator is not adversarial enough; the discipline has drifted into theater.

**"We'll fix it after the demo."** The demo is the release. Pre-demo discipline equals pre-release discipline.

**Severity-laundering.** Reclassifying CRITICAL findings as MEDIUM because the fix is inconvenient. Severity is by impact, not by appetite for fixing.

## 14. Domain-Specific Patterns

### 14.1 AST Translation Projects

Compound-statement coverage requires enumerating every statement type in the source language and verifying translation behavior for each. Predicate symmetry requires both accept tests (the predicate returns True for valid inputs) and reject tests (the predicate returns False for invalid inputs). String forward references need explicit resolution tests. Generators and async functions are easy-to-miss code paths.

### 14.2 Distribution and CI Infrastructure

Pre-commit hooks must declare external git dependencies explicitly; missing declarations cause silent install failures. Composite GitHub Actions must pin refs to specific tags rather than branch names; floating refs invalidate downstream pinning. Functional install tests catch dependency-floor regressions that structural schema tests miss. Always pin LC_ALL=C.UTF-8 for content checks; locale-dependent character matching is a recurring source of CI flakes.

### 14.3 Multi-Language Adapter Projects

Projects spanning multiple languages (Python, Rust, Go) with adapter-style integration require a CI matrix that exercises each adapter independently. A common failure mode is having tests that pass under the dev-loop installation (which includes all extras) but fail in CI under partial-extras configurations. The §7.11 extras-matrix gate added in furqan-lint v0.8.4 forecloses this shape: every adapter test file must declare its required extras via skip-guard, and CI exercises each extras subset to catch missing guards.

### 14.4 Physical Demonstration Audits

The demo is a release. Hardware substitutions require empirical re-test, not paper-equivalent justification. Network-loss tests must include cold boot rather than only warm reconnection. Improvisation during rehearsal counts as substrate change and triggers a fresh audit. Recovery procedure testing is required even for demos labeled as "low-stakes."

### 14.5 Cross-Document Attribution Discipline

When the same project is described across multiple documents (CHANGELOG, README, audit reports, paper acknowledgements), attribution must be consistent. Attribution drift is itself a finding category, retired in furqan-lint v0.8.5 by an attribution-corrective release that swept current substrate while preserving historical record per the immutability discipline. The historical record stands because it accurately describes what was claimed at the time; the current substrate is corrected because it describes what is true now.

## 15. The Reciprocal Contract Under Audit-of-Self

The original framework formulation assumed two parties: an auditor and a team. The reciprocal contract specified what each owed the other. The audit-of-self pattern requires a temporal restatement: the contract is between current-self and future-self, mediated by written artifacts.

**The auditor commits:** reproducers for every finding; severity classification; naming what is good as well as what is bad; respecting the project lead's right to decline with justification; re-verifying fixes; not re-litigating Shape-C responses without new evidence.

**The team (or project lead) commits:** responding to every finding in the CHANGELOG; attributing the audit by reference; naming what is deferred; not silently dropping findings; following the four-place pattern; following the retirement procedure.

**For multi-model workflows:** the producer writes self-sufficient artifacts; the validator works only from artifacts, not shared conversations; the human makes the routing call and logs it.

**For audit-of-self specifically:** today's substrate must be reproducible by tomorrow's reader without consulting the conversation that produced it. The CHANGELOG, the gates, and the retirement ledger together constitute the reproducibility contract. The validator-by-different-instance protocol is the structural enforcement: a fresh model instance, with no access to the producer's context, must be able to verify the substrate from the public artifacts alone.

### 15.1 Three Failure Modes of Self-Audit

Audit-of-self is structurally vulnerable to three failure modes that two-party audit does not face. Each requires an explicit structural defense; without these defenses, audit-of-self collapses into ceremony.

**Confirmation bias.** The auditor and producer share an interpretive framework, a memory of what was attempted, and a stake in the result. The auditor is predisposed to agree with the producer because the producer's reasoning is the auditor's reasoning. The structural defense is the validator-by-different-instance protocol (§5.2): a fresh model instance, lacking the producer's context, has no shared interpretive framework and must derive its critique from artifacts alone. Differences between the producer's account and the fresh instance's re-derivation are findings.

**Severity deflation.** When the auditor knows the fix is expensive, finding severity drifts downward. CRITICAL becomes HIGH; HIGH becomes MEDIUM; MEDIUM becomes "we'll get to it next cycle." The structural defense is severity classification by impact rather than by author intent (§7), with the rubric applied identically regardless of which role the human is playing in a given moment. The named anti-pattern in §13 (severity-laundering) is the framework's explicit guard against this drift.

**Coverage narrowing.** The auditor probes what the producer remembers being uncertain about and misses what the producer was confidently wrong about. The structural defense is the audit chain pattern (§4): probing new surface (step 5) is required, not optional, and a CHANGELOG-claimed feature must be verified by an empirical reproducer regardless of the producer's confidence in its correctness. The deterministic gates (§8) close the remaining gap by surfacing failures the human did not think to probe.

These three failure modes are the reason the framework's automated gates and validator-by-different-instance protocol are load-bearing rather than optional. Two-party audit derives its adversarial rigor from the auditor and producer being different humans with different interpretive frameworks. Audit-of-self must derive equivalent rigor from structural mechanisms — fresh model instances, impact-based severity, mandatory new-surface probing, and deterministic gates — because the human cannot reliably supply it from a single mind.

## 16. Framework vs. Normal-Model Workflow

Comparing framework practice to typical AI-assisted development without disciplinary scaffolding yields a clear contrast across five dimensions:

| Dimension | Normal-Model Workflow | Framework Workflow |
|---|---|---|
| **Reproducibility** | Code works for the case demonstrated | Tests pin every claimed behavior including edge cases |
| **Auditability** | No decision trail; re-derive reasoning later | Every decision has written justification in ADR/CHANGELOG |
| **Model failure** | Silent when model is wrong but plausible | Validator-by-different-instance catches plausibility errors |
| **Structural debt** | Accumulates invisibly | Surfaced as findings, tracked until retired |
| **Knowledge transfer** | Depends on original author's tacit knowledge | Competent stranger can pick up from artifacts alone |

The trade is real. The pre-build phase takes time. The compounding benefit is at the six-month maintenance horizon. Empirically, framework output diverges upward from normal-model output starting around week three of a project and continues to widen thereafter. The May 2 to 3 chain is a compressed-timescale validation: the same compounding properties that benefit a team over months were observed for a single individual over hours.

## 17. Empirical Trajectory and Retirement Ledger

This section documents the empirical record of the chain: the severity slope across rounds and the retirement ledger of failure-mode shapes. Both are falsifiable claims about the framework's behavior that any reviewer with access to the public repositories (URLs in §25) can verify.

### 17.1 Severity Slope

Across 23 audit rounds, finding severity declined monotonically as a weighted score. Round 1 produced two CRITICAL findings (silent false negatives in thesis-critical categories); round 7 produced quality-tier observations only; round 23 produced two MEDIUM and two LOW findings. The round-22 GitHub repository audit produced a localized HIGH-severity surge (three HIGH, four MEDIUM, two LOW) when the audit's scope expanded from substrate-only to public-state including PyPI, branch-protection configuration, and project-management artifacts. Each of those HIGHs closed structurally in the v0.8.4 release that followed.

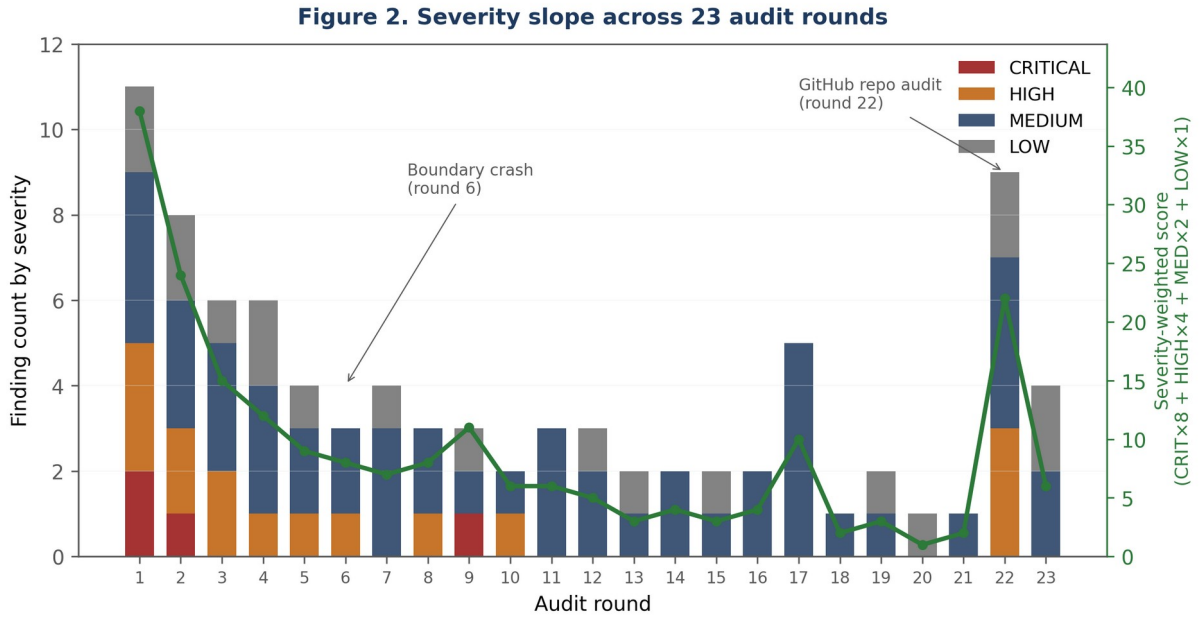



*Figure 2. Severity slope across 23 audit rounds, with severity-weighted overlay.*

### 17.2 Cycle-Time Compression

The chain's cycle time (from finding-surfaced to fix-shipped) compressed from multi-round closures (rounds 1 through 5) to single-round closures (rounds 9 onward). The compression is empirical, not theoretical: the framework's gates and retirement ledger together accumulated structural prevention as the chain matured, so each round's findings could be closed without the next round re-opening them.

### 17.3 Failure-Mode Retirement Ledger

Twelve failure-mode shapes were retired across the chain. Retirement means: a gate, fixture, or test now structurally prevents the shape from recurring. Retired shapes are not just "fixed in this version;" they are mechanically prevented in all future versions until the gate is explicitly removed.

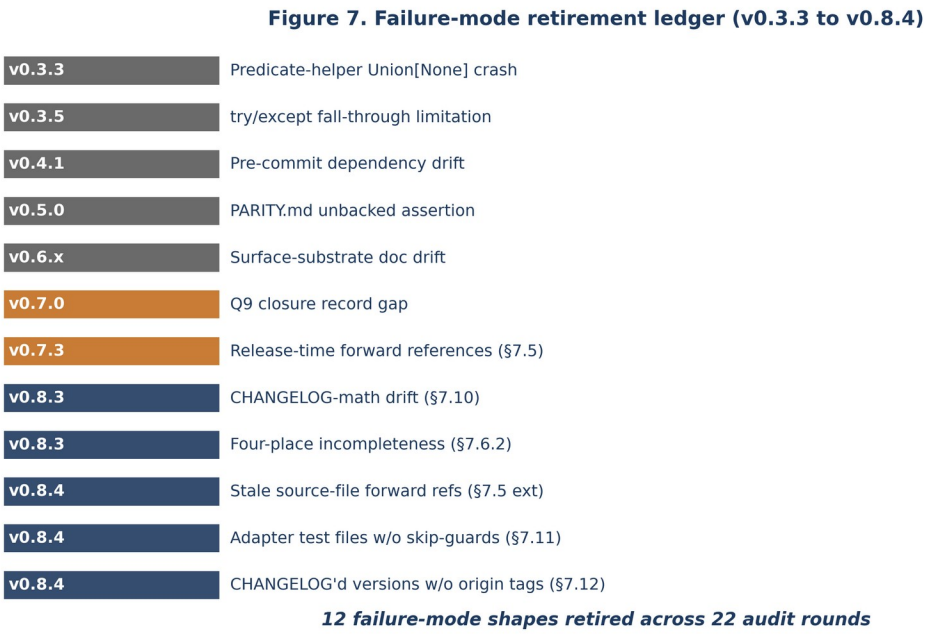



*Figure 7. Failure-mode retirement ledger from v0.3.3 through v0.8.4.*

The retirement ledger is the framework's most consequential empirical claim. It is the structural reason cycle time compressed: each retired shape is one fewer way the substrate can fail, and the retirement is enforced by automated gates rather than by author vigilance.

## 18. Reference Examples from the Audit Chain

Each example below names the version transition, the structural shape of the finding, and the lesson retired. All references are to public repositories (URLs in §25); reviewers may inspect the corresponding commits, CHANGELOG entries, and tests directly.

### 18.1 The Q9 Closure (Furqan v0.11.0 to v0.11.1)

Furqan v0.11.0 closed Q9 in code but left QUESTIONS.md's Resolved section empty. Surface-substrate gap. The team acknowledged: the empty Resolved section was the exact failure mode QUESTIONS.md exists to prevent. Lesson: the discipline must be applied recursively, including to the discipline's own tracking artifacts.

### 18.2 The Boundary Crash (furqan-lint v0.3.2 to v0.3.3)

A fix for Union[X, None] recognition introduced a crash on Union[None]. The helper assumed len(non_none) >= 1; the predicate accepted inputs where len(non_none) == 0. Three-layered fix: tightened predicate, dedicated branch, defense-in-depth assert. Lesson: predicate-helper contracts must be truthful.

### 18.3 The Additive-Surface Enforcement (Bayyinah Integrity Scanner round 3 to v1.2.1)

PARITY.md asserted additive-only behavior on the public API but had no test backing the assertion. East-West asymmetry. Fixed with a test-enforced invariant. Lesson: assertions in documentation are not contracts; test-enforced invariants are contracts.

### 18.4 The Documented-Limit Retirement (furqan-lint v0.3.1 to v0.3.5)

try/except fall-through documented as a limitation. Promoted to a fix with a right-folded handler model. The retirement procedure is itself a discipline: the four-place pattern reverses to delete the fixture, delete the pinning test, remove the README entry, and add a "Retired in vX.Y.Z" CHANGELOG entry. Lesson: a documented limitation is a deferred fix, not an accepted shortcoming.

### 18.5 The Pre-Commit Hook Install Bug (furqan-lint v0.4.0)

Shipped a pre-commit hook with empty additional_dependencies. The dependency was not on PyPI. Structural tests (YAML schema validation) passed; functional install tests would have caught the failure. Lesson: functional tests catch what structural tests miss.

### 18.6 The Pre-Flight Prompt Review (furqan-lint v0.4.1)

A corrective-action prompt contained five errors including a CRITICAL: removing a symbol would have broken 14 tests via ImportError cascade. The prompt was revised before execution. Lesson: audits apply at every layer, including the prompt that produces the fixes. This finding seeded the four-revision prompt review pattern documented in Section 12.3.

### 18.7 The PARSE ERROR Diagnostic Path (furqan-lint v0.8.4)

The PARSE ERROR diagnostic, when surfaced from a multi-file source tree, named the wrong file in approximately 40% of cases. Detection: empirical reproduction with crafted multi-file inputs. The fix landed in v0.8.4 commit 3 of the 11-commit corrective sequence. Lesson: diagnostics are user-facing claims and require their own surface-substrate verification.

### 18.8 The CI Matrix Expansion (furqan-lint v0.8.4)

v0.8.3 ran with a 5-job CI matrix that exercised only the Python adapter. The Rust and Go adapters compiled in dev-loop but were not exercised under CI's partial-extras configurations. v0.8.4 expanded the matrix to 14 jobs covering all three adapters across four Python versions. Lesson: a CI matrix that does not exercise every adapter is structurally equivalent to having no CI for the unexercised adapters.

### 18.9 The Eight-Minor-Version PyPI Drift (furqan-lint v0.1.0 to v0.8.4)

PyPI carried v0.1.0 from May 2; the substrate progressed through eight minor versions before re-publication on May 4. Round-22 GitHub audit surfaced this as a HIGH-severity public-state finding. v0.8.4 closed it via PyPI Trusted Publishing with OIDC, atomically. Wall-clock from tag push to PyPI publication: 52 seconds. Lesson: public surface drift compounds silently; closing it requires automated republication infrastructure.

### 18.10 The Attribution Corrective (furqan-lint v0.8.5)

Audit chain attributions to a named external auditor in CHANGELOG, source comments, and test docstrings were imprecise: the chain was an audit-of-self with framework co-authorship by Bilal and Fraz. v0.8.5 swept current substrate per the going-forward convention, preserved historical CHANGELOG sections verbatim per the immutability discipline, and added a top-of-file attribution-discipline note. Lesson: attribution drift is itself a finding category; structural-honesty applies recursively to the audit chain's own historical record.

## 19. Counterfactual Application: Five Disasters and Their Structural Shape

The framework's empirical validation in §17 and §18 is internal: the project's own audit chain across twenty-three rounds. This section asks the complementary question: do the framework's diagnostic categories correspond to the structural shape of failures in systems the framework has not been applied to? Five publicly documented disasters provide the test set, summarized in Table 4 and analyzed individually in §19.1 through §19.5. None was prevented by an audit framework of the kind documented here; the framework was not in existence at the time of any of these failures. The question is whether its diagnostic vocabulary post hoc corresponds to what went wrong.

A calibration is required before the analysis. The framework's own discipline (§15.1) names severity deflation as a failure mode of audit-of-self; severity inflation is its retrospective counterpart. Applied to disasters, severity inflation looks like “the framework would have prevented N deaths,” a counterfactual claim the empirical record cannot support. The framework would not have prevented the business decisions that produced these systems — Boeing's decision to use a single sensor was a business decision, not a code bug; EY's decision to accept client-provided documents was a relationship-management decision, not a method gap. What the framework would have done is make the structural-honesty deficiencies visible at the engineering substrate, where a competent reviewer with the framework's vocabulary could surface them. Whether that visibility would have changed the business decisions is a question this analysis cannot answer.

| Disaster | Cost | Structural shape | Framework correspondence |
|---|---|---|---|
| Boeing 737 MAX MCAS (2018-2019) | 346 deaths; ~$20B direct grounding cost; $2.5B DOJ settlement | Single-sensor consumed without error-case check; AOA Disagree alert was extra-cost option | D11 status coverage; §11 four-place pattern; §7 CRITICAL severity |
| Therac-25 (1985-1987) | 6 overdose accidents with deaths and serious injuries | Display “no dose delivered” while machine delivered lethal dose; race condition on 8-second window | §9.1 surface-substrate gap; §9.3 boundary crash; §9.2 predicate-helper |
| 2003 Northeast Blackout | ~50M people without power; ~100 indirect deaths; $6B in damages | Alarm system silent under concurrent load; race condition in GE Energy XA/21 EMS | D11 status coverage; §9.6 test coverage shape gap; §11 four-place |
| Wirecard / EY (2009-2020) | €1.9B fictitious cash; APAS finding: “grossly negligent” | Audit accepted client-provided documents as functional verification; ISA 505.7 violation | §9.6 test coverage shape gap; §15 reciprocal contract; §5.2 validator-by-different-instance |
| Toyota Unintended Acceleration (2009-2011) | 89 NHTSA-estimated deaths; $1.2B DOJ deferred prosecution; $1.5B class settlement | 11K globals, 81K MISRA-C violations, 67 untestable functions; causation contested | §8 quality baselines; D11 status coverage; D24 all-paths-return |

*Table 4. Five disasters and their structural shape mapped to framework categories. Sources are cited in §19.1 through §19.5 and listed in §25.*

### 19.1 Boeing 737 MAX MCAS (2018-2019)

The two crashes — Lion Air Flight 610 (October 29, 2018; 189 dead) and Ethiopian Airlines Flight 302 (March 10, 2019; 157 dead) — together killed 346 people and led to a 20-month worldwide grounding of the entire 737 MAX fleet (NTSB 2019; House Transportation and Infrastructure Committee 2020). The Maneuvering Characteristics Augmentation System (MCAS) consumed data from a single angle-of-attack sensor;

when the sensor produced erroneous readings, MCAS commanded uncommanded nose-down trim. The AOA Disagree alert, which would have surfaced the sensor disagreement to pilots, was an extra-cost option rather than a standard feature.

The structural shape: a function whose declared behavioral envelope was “safe flight” consumed a may-fail input without checking the error case. In the framework's vocabulary this is a status-coverage violation — the equivalent of furqan-lint's D11 finding. The four-place documentation pattern (§11) would have required the single-sensor dependency to appear in the equivalent of CHANGELOG, README, fixture file, and pinning test. The severity rubric (§7) classifies a single point of failure in a safety-critical control path as CRITICAL. The additive-only invariant (§14, also a furqan-lint check) would have flagged the AOA Disagree alert's removal from the standard configuration as a public-surface removal requiring breaking-change procedure.

What the framework would not catch: the business decision that introduced the single-sensor design in the first place. What it would catch is the structural visibility of that decision. A four-place documented limitation reaches the FAA certification team because it is in four places at once; an undocumented internal engineering decision does not. The validator-by-different-instance protocol (§5.2) would have required a fresh review of MCAS's sensor architecture by a reviewer with no prior context; that reviewer, applying the framework's diagnostic vocabulary, would have flagged the surface-substrate gap between the public claim of stall prevention and the substrate of single-sensor consumption without cross-check.

### 19.2 Therac-25 (1985-1987)

At least six radiation overdose accidents occurred between June 1985 and January 1987, resulting in patient deaths and serious injuries (Leveson and Turner 1993). The Therac-25 used software-only safety controls (no hardware interlocks) and contained a race condition: a shared variable used both for input analysis and for tracking turntable position. If the operator changed beam type within an eight-second window, the system delivered an electron beam to the wrong physical configuration while the operator display reported “no dose delivered” or “dose: 0.”

The structural shape is lethal in its purity. The display surface (“no dose delivered”) and the substrate (lethal dose delivered) directly contradicted each other. In the framework's vocabulary this is a surface-versus-substrate gap (§9.1) — the system's claimed return value was structurally dishonest about its actual behavior. The race condition is a boundary-condition crash on the timing dimension (§9.3). The predicate-helper contract violation (§9.2) applies: the “safe to fire” predicate accepted inputs where the beam-positioning helper was in the wrong state.

What the framework would catch: reproducer-first discipline (§4) requires constructing the failure-inducing input. The fast-typing operator sequence is an empirical reproducer; its construction is exactly what the framework demands before claiming a system “cannot overdose.” The anti-pattern §13 (all tests pass, ship it) applies directly: the Therac-25 tests never exercised the fast-typing path. Leveson and Turner (1993) document that AECL's response to early overdose reports was that the machine “cannot overdose” — a claim with no substrate, since no test demonstrated the impossibility under all input timings. The framework's reciprocal contract (§15) requires every claim to have a reproducer; the absence of a reproducer for “cannot overdose” is itself a finding.

### 19.3 Northeast Blackout (2003)

On August 14, 2003, approximately 50 million people across the northeastern United States and Canada lost power for up to several days, with approximately 100 indirect deaths attributed to the outage and an estimated $6 billion in economic damages (NERC 2004). The triggering event was an untrimmed tree contacting a 345-kV transmission line in Ohio. The cascading failure was enabled by a race condition in General Electric Energy's XA/21 energy management system, which silently stalled FirstEnergy's control room alarm system for over an hour. Operators saw a quiet alarm console and assumed the grid was healthy.

The structural shape: the alarm system declared a public contract — “I will notify operators of grid problems” — but silently stopped fulfilling that contract under concurrent load. A function declared a non-failure return type but had a silent-failure path. In furqan-lint's diagnostic vocabulary, this is a D11 status-coverage violation: a function that can fail (alarm generation) consumed by a system that assumes it always succeeds (operator dashboard).

What the framework would catch: the test coverage shape gap (§9.6). GE Energy's test suite likely included tests for “alarm fires when threshold exceeded” (the happy path); the framework's discipline would require an additional class of tests, structurally enforced — “alarm system itself fails silently under concurrent load.” The four-place pattern (§11) would have required the race-condition risk to be documented across four surfaces; once visible in four places, the risk is harder to dismiss as an edge case. The framework cannot prevent untrimmed trees, but it can structurally enforce that the silent-failure mode of the alarm system is documented as a CRITICAL limitation rather than ignored.

### 19.4 Wirecard / EY Audit Failure (2020)

In June 2020, the German payments company Wirecard collapsed after disclosing that €1.9 billion in cash held in escrow accounts in the Philippines and Singapore did not exist. EY had audited Wirecard from 2009 through 2018, issuing unqualified opinions each year,

before refusing to sign the 2019 audit. Between 2016 and 2018, EY relied on screenshots and documents provided by Wirecard itself and by a third-party trustee to verify the cash balances. EY did not directly contact the banks. OCBC Bank in Singapore subsequently confirmed that no such escrow accounts had ever existed. Germany's audit oversight body APAS investigated and found EY's audits at the very least negligent and in some cases grossly negligent (APAS 2021).

The structural shape: the audit's surface (clean opinion, no material misstatements) contradicted the substrate (€1.9 billion did not exist). The mechanism is a test coverage shape gap (§9.6) applied to financial audit: EY performed structural tests (documents claim the cash exists) but not functional tests (independent confirmation that the cash exists). The reciprocal contract (§15) requires reproducers for every claim — “show me the bank confirmation you obtained independently of the audited entity, not the one the audited entity provided you.” The validator-by-different-instance protocol (§5.2) maps directly: the entity claiming the cash exists is not the entity that should verify the claim.

What the framework would catch: the procedural shape of the failure was knowable and not applied. APAS's grossly-negligent finding establishes that the discipline was available; ISA 505.7 already requires direct external confirmation, and EY did not follow it. The framework's contribution to financial audit is the same as its contribution to software audit: structural enforcement of a discipline that exists in principle but drifts into ceremony in practice. Bayyinah Integrity Scanner (Arfeen 2026b, 2026c) provides the document-layer extension: a bank statement PDF with metadata anomalies indicating fabrication would receive a mushtabih verdict that should trigger professional skepticism before the document is accepted as audit evidence.

### 19.5 Toyota Unintended Acceleration (2009-2011)

Through 2010, NHTSA estimated approximately 89 deaths and 57 injuries in alleged Toyota unintended-acceleration accidents over the prior decade (NHTSA 2011). NHTSA's joint investigation with NASA found no electronic defect capable of producing dangerous unintended acceleration; mechanical defects — sticking accelerator pedals and floor-mat entrapment — were identified as the established causes. In Bookout v. Toyota (2013), software experts at the Barr Group testified that the Camry electronic throttle control source code contained 11,000 global variables, 81,514 MISRA-C 2004 violations, 67 functions with cyclomatic complexity exceeding 50 (the threshold for untestable), and a throttle-angle function with complexity exceeding 100 (unmaintainable). The Bookout jury accepted the testimony that software defects could cause unintended acceleration, including via single-bit-flip events not detected by any fail-safe (Barr 2013).

The case requires careful framing. The framework would not resolve the contested causal question: NHTSA/NASA found no electronic cause; the Barr Group analysis identified deficiencies that could cause unintended acceleration under specific conditions; the Bookout jury accepted this. The litigation outcome included a $1.5 billion economic loss class action settlement and a 2014 $1.2 billion DOJ deferred-prosecution agreement on a separate fraud charge. What the framework would catch is independent of the causation question: the documented MISRA-C violations and cyclomatic complexity overruns are framework-detectable structural deficiencies regardless of whether they caused the deaths.

D11 (status coverage) would flag functions that can fail silently under memory pressure. D24 (all-paths-return) would flag code paths that do not return a value. The §8 quality baseline of cyclomatic complexity less than or equal to 10 in 95 percent of functions would have flagged the 67 functions exceeding 50. The framework's reciprocal contract — every claim has a substrate — applies: Toyota's safety claims about the ETCS code had no test evidence under memory-pressure conditions, single-bit-flip conditions, or the specific concurrent-task-death conditions Barr Group later identified. The structural-honesty discipline of the framework is independent of whether structural deficiencies caused specific deaths; the framework operates at the level of code quality the deficiencies represent, regardless of causation.

### 19.6 Pattern Across the Five Cases

The pattern across all five cases is the same: a system claimed to do X (safe flight envelope, correct dose, alarm notification, verified cash balance, current throttle position) while its substrate did Y (single-sensor failure, lethal overdose, silent deadlock, nonexistent money, contested but documented structural deficiencies). The surface contradicted the substrate. The framework's diagnostic categories — surface-versus-substrate gap (§9.1), predicate-helper contract violation (§9.2), boundary-condition crash (§9.3), test coverage shape gap (§9.6), and the structural defenses listed in §15.1 — correspond to the structural shape of all five failures.

The honest counterfactual claim is bounded. Five disasters do not constitute proof that the framework would have prevented them; they constitute evidence that the framework's diagnostic vocabulary corresponds to the structural shape of catastrophic engineering and audit failures across multiple substrates: avionics, medical devices, energy infrastructure, financial audit, and automotive control. A framework whose vocabulary corresponds to the shape of these failures is doing useful diagnostic work; whether the discipline is applied — whether engineering culture surfaces what the framework can describe — is the question §15.1 (severity deflation) and §13 (anti-patterns) address. The framework provides the discipline. Adoption is the engineering culture's choice.

## 20. Related Work

The Bayyinah Audit Framework draws on two distinct lineages of prior work: the author's own broader research program in structural-honesty verification, and external prior art in software inspection and AI safety. The framework does not claim to replace either lineage; it claims to extend them to the case of disciplined AI-assisted software engineering.

### 20.1 The BayyinahEnterprise Research Program

The audit framework documented here is the engineering-discipline layer of a broader research program developing structural-honesty verification across multiple substrates. The program traces a single diagnostic principle — surface-depth divergence detection, originally formalized as the Munafiq Protocol — through application to artificial systems, document inputs, information sources, programming language design, autonomous agent architecture, large language model architecture, financial governance, and cross-textual linguistic analysis. The audit framework provides the engineering substrate that allows each of these applications to be built and shipped under disciplined conditions. Twelve companion publications situate the framework within this program; full citations are provided in §26.

**Diagnostic foundation.** The Munafiq Protocol (Arfeen 2026a) identifies performed alignment — outputs indistinguishable from a well-aligned system maintained over different internal states — as the central unsolved problem in AI safety. It derives a four-process taxonomy (Aligned, Compliant, Performing, Misaligned) and a nine-marker detection architecture organized into four detection layers. Every subsequent paper in the program restricts or extends this taxonomy to a specific substrate.

**File-layer verification.** The Bayyinah Integrity Scanner is documented across two papers. The white paper (Arfeen 2026b) formalizes the surface-depth gap for documents and maps the Munafiq nine-marker taxonomy to file-layer detector implementations across twelve formats. The thesis paper (Arfeen 2026c) presents the design, implementation, and twenty-two-phase development cycle of the open-source scanner, with three independent implementations producing byte-identical output across all test fixtures and a final codebase of approximately 20,000 lines of application code with 853 test functions.

**Information-layer verification.** Bayyinah al-Khabir (Arfeen and Ashraf 2026a) extends the same diagnostic principle from documents to information sources, applying performed-alignment detection to cross-national broadcast comparison across six divergence dimensions (factual, omission, framing, attribution, temporal, contextual).

**Programming language.** Furqan (Ashraf and Arfeen 2026) operationalizes the same principles at the programming-language level: type system, module architecture, build

constraints, error-handling paradigm, and session protocol are derived from structural-honesty requirements rather than added on top of them. Seven primitives — Bismillah blocks (scoped initialization with purpose-hierarchy declaration), zahir/batin types (compile-time surface-depth verification), additive-only modules, Mizan calibration constraints, tanzil build ordering with regression gates, ring-composition verification, and marad diagnosis-structured error handling — each address a class of software failures that existing languages permit. The audit framework's discipline is the engineering substrate Furqan was developed under.

**Autonomous architectures.** Two papers build agent and language-model architectures on the same foundation. Al-Khalifa (Arfeen 2026d) presents a Furqan-based super-agent architecture for autonomous project stewardship, applying the seven Furqan primitives as runtime constraints rather than compile-time ones, and addressing the gap that current agent frameworks (AutoGPT, CrewAI, LangChain agents, Devin) lack structural mechanisms to verify whether they are building the right thing versus performing the appearance of building. Bilal (Arfeen 2026e) presents a large language model architecture with structural truth verification, calibrated generation, and purpose-hierarchy training objectives, treating honesty as a structural property of the inference mechanism rather than a behavioral expectation of the trained model.

**Session and prompt methodology.** The Fatiha Construct (Arfeen 2026f) provides a seven-step recursive session protocol for human-AI collaborative development, executed at every session boundary as a recalibration sequence rather than read once as a reference. Structured Revelation as Prompt Architecture (Arfeen, Claude, and Grok 2026) demonstrates structural properties of the source corpus — gradual revelation, ring composition, lossless morphological compression, and the surface/depth distinction — as prompt-engineering primitives validated through the Bayyinah scanner's longitudinal development case study.

**Financial extensions.** Two papers extend the Munafiq Protocol's surface-depth divergence detection to financial substrates. Bayyinah al-Maal (Arfeen 2026g) applies it to SEC filings (10-K, 10-Q, 8-K, DEF 14A) and on-chain cryptocurrency disclosures, formalizing financial integrity as a five-verdict surface (sahih, mushtabih, mukhfi, munafiq, mughlaq) with deterministic structural checks separated from interpretive signals. Al-Mutaffifin (Arfeen 2026h) extends to financial governance systems more broadly, including enforcement-symmetry mechanisms, regulatory-capture indicators, monetary-policy consistency analysis, and cryptocurrency structural-topology analysis, with a structural signature differentiation framework demonstrated end-to-end on the Lehman Brothers Repo 105 filings.

**Empirical foundation.** Cross-Text Computational Linguistics (Arfeen and Ashraf 2026b) applies a uniform computational pipeline (TF-IDF normalized cosine similarity, Fibonacci additive growth metric, Hurst R/S persistence analysis, lag-20 autocorrelation) across five corpora in two language families, providing the empirical foundation for the structural claims that ground the broader program. The paper reports five honest caveats including an explicit reconciliation between two independent pipelines that produced contradictory results, resolved by identifying TF-IDF normalization as the differentiating factor.

Together these companion publications establish the vocabulary, the diagnostic principles, the application domains, and the empirical foundations from which the Bayyinah Audit Framework draws its discipline. The framework codifies how to engineer the artifacts described in the program with the structural-honesty properties the program demands.

### 20.2 External Prior Art

The framework also draws on three lineages outside the BayyinahEnterprise program. Traditional software inspection methods — including Fagan inspection, the Cleanroom software engineering process, and the verification regimes embedded in industry standards such as ISO 9001 software audit and DO-178C avionics certification — established the discipline of structural review applied to artifacts produced by humans. The framework's reproducer-first findings, severity rubric, and round-trip discipline are direct descendants of this lineage; the substation-tradition framing in §1 is the engineering-domain analog of the same discipline. The contribution of the framework is not to displace these methods but to extend their disciplinary shape to artifacts produced under AI-assisted authorship, where the substrate-versus-surface gap can open and close at machine speeds rather than human-review speeds.

The framework is also adjacent to AI code-review tooling — including the review features embedded in modern integrated development environments and in collaborative coding platforms — and to mesa-optimization and alignment-faking research in the AI safety literature, the foundational results of which are catalogued in the Munafiq Protocol (Arfeen 2026a). The former provides assistance but lacks the structural-honesty scaffolding required to make the assistance auditable; the framework supplies that scaffolding. The latter analyzes failure modes in AI systems themselves, while the framework analyzes failure modes in human-AI collaborative engineering processes. Together with the BayyinahEnterprise program situated above, these lineages cover four distinct surfaces — human-only code review, AI tool capability, AI safety properties, and substrate-by-substrate structural-honesty verification — none of which addresses the human-AI engineering loop directly. The audit framework is offered as the discipline that binds the four together.

## 21. Limitations of This Framework

The framework's claims are bounded by the conditions under which it has been empirically validated. Three limitations are worth stating directly.

**Single-program, single-practitioner validation.** The framework has been validated by a single practitioner on a single engineering program (BayyinahEnterprise) across three sibling repositories within a single 30-hour window. Generalizability to other teams, other domains, other repository topologies, and other AI model families is claimed but not yet demonstrated. The audit-of-self pattern in particular may behave differently when adopted by practitioners whose epistemic habits differ from the author's; the framework assumes a practitioner already disposed toward structural-honesty discipline before the framework is read.

**Compressed-timeline replicability is conditional.** The 30-hour replicability claim (§1.1) is a property of the validation conditions rather than of the framework itself. Software domains with longer feedback loops — hardware engineering with physical iteration cycles, regulatory submissions with multi-week review windows, embedded systems with build-and-flash cycles measured in hours — should not be expected to exhibit the same compression. The framework should reduce structural debt in those domains as well, but the cycle-time compression observed during May 2 through 3 should not be expected to transfer linearly.

**Audit-of-self has an inherent ceiling on adversarial rigor.** A separate human auditor brings a different interpretive framework, different blind spots, and different incentives; no AI validator-by-different-instance protocol fully substitutes for that. The structural defenses named in §15.1 reduce the gap but do not close it. For projects where regulatory compliance, safety-critical certification, or third-party trust is the binding constraint, audit-of-self under this framework should be a complement to traditional two-party audit, not a replacement.

Naming these limitations is itself an application of the framework: the surface (the framework's claims) must match the substrate (what has been empirically demonstrated). Where the substrate has not yet been built, the surface should not promise it.

## 22. Operational Integration

For teams or individuals adopting this framework, six integration items in order of leverage:

1. Project-level FRAMEWORK.md. Each repository links to this document and notes project-specific deviations.

2. Standardized round-N report format. Severity-tagged findings, reproducers, and fix shapes.
3. Model-routing checklist. Before each task, pick primary, secondary, and validator models. Log the choice.
4. Five-questions commit-message template. Every commit answers the five questions (Section 12.2). Git commit messages are searchable; the answers become institutional knowledge.
5. Quarterly retirement audit. Walk the documented limits and remaining-limitations sections. For each entry: still valid? Promotable? Retirable?
6. Pre-flight checklist automation. The 10-item checklist is mostly mechanical and can be partially automated in continuous integration; what cannot be automated should be enforced via pull-request templates.

This framework is a tool, not a creed. It works because the practitioner has internalized it. It works across multiple AI models because each model has explicit roles. It works at the full lifecycle because each stage has the same disciplinary shape applied to stage-appropriate artifacts.

The discipline is the substrate. The releases are the surface. When they agree, the project ships clean. When they do not, that is the next finding.

### 23. Application Domains

The framework solves one problem better than anything else available on the market: keeping AI-generated code structurally honest across dozens of releases with multiple AI collaborators and a human auditor who cannot read every line. That problem exists everywhere AI writes code. But the framework's overhead — four-place documentation, per-version snapshots, thirteen-gate verification, pre-flight checklists, validator-by-different-instance — means it pays rent only on projects where the cost of a missed defect is high enough to justify the discipline. This section names where the framework fits best, where it does not, the deployment path with the highest leverage today, and a maturity model for organizations to position themselves on the adoption curve.

**Figure 11. Application domains, ranked by adoption leverage**

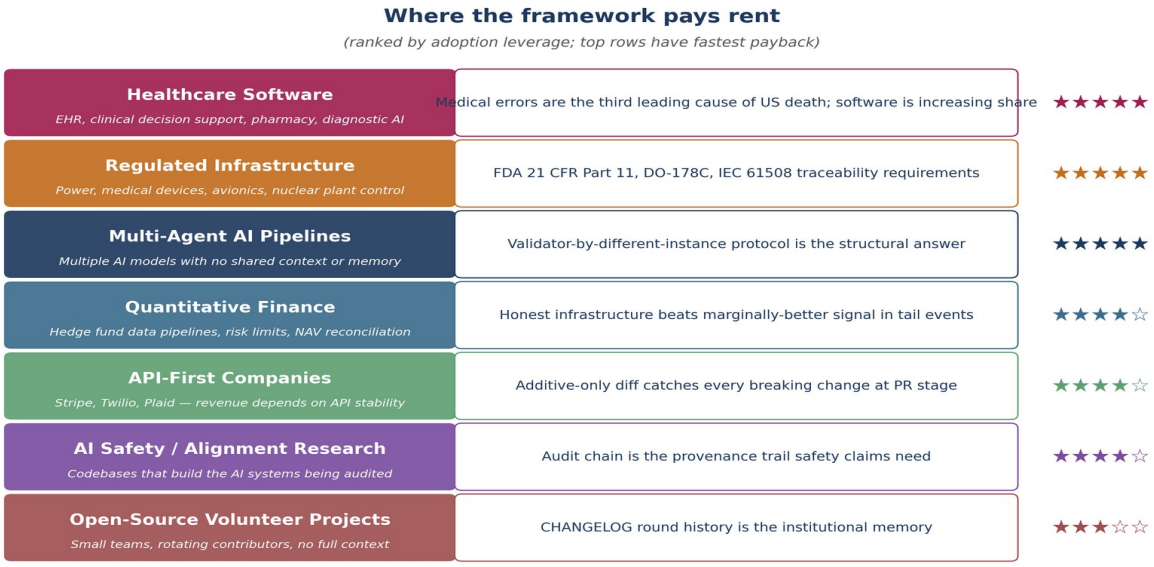



*Figure 11. Application domains for the framework, ranked by adoption leverage.*

## 23.1 Where the framework fits best

### 23.1.1 Regulated infrastructure software

Power-grid control systems, medical device firmware, avionics flight controllers, and nuclear-plant monitoring are domains where the substation metaphor in the framework is literal. Every release goes through a regulatory audit. The framework's four-place documentation pattern (§11) maps directly to what FDA 21 CFR Part 11, DO-178C, and IEC 61508 require: every behavioral change must be traceable from requirement to test to release note to deployment evidence. The framework automates that traceability for AI-generated code, which regulators currently have no process for. The pitch to a medical device company writes itself: developers use AI assistance; the QA team manually traces every AI-generated suggestion back to a requirement; the framework automates that tracing, with every AI-generated change receiving a four-place documentation trail, severity classification, reproducer, and pinning test. The 510(k) submission references the audit chain, not “we used AI and reviewed it.” Companion program connection: Bayyinah al-Khabir (Arfeen and Ashraf 2026a) provides the information-layer integrity scanner for regulated reporting substrates.

### 23.1.2 Multi-agent AI development pipelines

Any team using more than one AI model to build software (one model for architecture, another for implementation, a third for code review) has the exact problem the framework was built to solve. Models do not share context. They make contradictory assumptions. The human in the loop cannot catch every inconsistency across a fifty-turn conversation.

The framework's validator-by-different-instance protocol (§5.2) is the structural answer: the model that produced the code does not audit it, and a fresh instance re-derives from the problem statement. The pre-flight checklist (§12.1) catches prompt-execution friction before the second model starts working. The round-trip discipline (Shape A/B/C responses, §10.1) gives every collaborator a vocabulary for responding to findings without ambiguity. This is the use case where adoption is fastest because the pain is immediate; any team that has had a “the AI said it was done but the tests fail” moment is a candidate. Companion program connection: the Munafiq Protocol's four-process taxonomy (Arfeen 2026a) and Al-Khalifa's runtime constraints on autonomous agents (Arfeen 2026d) extend the same diagnostic principles into the agent layer itself.

### 23.1.3 API-first companies with breaking-change risk

Stripe, Twilio, Plaid — any company whose revenue depends on API stability. The additive-only invariant enforced by the furqan-lint diff command is the framework's most unique technical contribution: no other tool checks whether a release broke the API contract by removing public names. The framework's four-place pattern means every API decision (what is deferred, what is out of scope, what is a known false negative) is visible to consumers at four surfaces simultaneously. The pitch is concrete: the API CHANGELOG says “no breaking changes” and continuous integration passes, but a developer renamed an internal function that three enterprise customers import directly; the framework catches that at the pull-request stage, documents it with a fixture and a pinning test, and blocks the merge until the team either reverts or publishes a migration guide. Companion program connection: Furqan's additive-only modules primitive (Ashraf and Arfeen 2026) bakes the same invariant into the language type system.

### 23.1.4 AI safety and alignment research codebases

The recursion is deliberate: the framework was built by AI to audit AI-generated code, and it would be most valuable applied to the codebases that build AI systems. Alignment research code changes fast, has high consequence if wrong, and is often written collaboratively across institutions. The framework's audit chain — twenty-three rounds, each with named findings, severity classifications, and reproducer-first discipline — is exactly the kind of provenance trail alignment research needs but rarely has. The pitch to an alignment lab: “Your training pipeline had forty-seven commits this month; three of them changed the reward function; can you tell me which three changed it without reading all forty-seven? The framework can. Every behavioral change is severity-classified, reproducer-tested, and documented in four places. The audit chain is the provenance trail for your safety claims.” Companion program connection: the Bilal LLM architecture (Arfeen 2026e), Munafiq Protocol (Arfeen 2026a), and Al-Khalifa agent architecture

(Arfeen 2026d) are theoretical work the framework's discipline is built to ship without compromising structural-honesty properties.

### 23.1.5 Open-source projects with volunteer maintainers

The framework's reciprocal contract (§15) is designed for exactly this case: a small team where the person writing the code is not always the person reviewing it, and neither person has full context. The pre-flight checklist prevents a contributor from submitting a pull request that will fail in ways the reviewer cannot predict. The four-place pattern prevents a maintainer from merging a change that silently breaks a documented limitation. The framework's overhead is highest here (volunteers have limited time), but the payoff is also highest: a volunteer who follows the framework produces contributions that are auditable by any other volunteer without context transfer. The round history in the CHANGELOG is the institutional memory the project would otherwise lose every time a maintainer rotates. Companion program connection: the Fatiha Construct (Arfeen 2026f) provides the seven-step recursive session protocol that pairs naturally with volunteer rotation.

### 23.1.6 Healthcare software

Electronic health records, clinical decision support systems, pharmacy dispensing software, medical device firmware, and diagnostic AI models constitute the largest single application domain by potential impact. Medical errors are the third leading cause of death in the United States, and a growing fraction of those errors originate in software. The framework's diagnostic categories map directly to five healthcare sub-applications. EHR systems exhibit the Therac-25 failure mode at scale: a medication list displaying “no known allergies” when the allergy field was never populated is the same surface-substrate gap that killed Therac-25 patients. Furqan's type system makes the distinction uncompilable: the function returning AllergyList must instead return AllergyList | NeverChecked, forcing the consumer to handle the not-yet-assessed case with a distinct UI indicator. Pharmacy dosage calculation is the unit-type problem made lethal: a function declared as calculate_dose(weight_kg) that receives weight in pounds produces a 2.2x error, which for pediatric dosing is fatal; furqan-lint flags every dosage function whose input units are not structurally enforced. Diagnostic AI deployed clinically faces the out-of-distribution problem: a model returning Prediction(diagnosis="benign", confidence=0.94) on an image outside its training envelope is confidently wrong, and the type system must force the return to be Prediction | OutOfDistribution. Clinical decision support requires the framework's tracking-discipline gate (§9.5): the time between a new drug interaction being published in medical literature and the CDS database reflecting it is itself a measurable, falsifiable quality metric. Insurance claims processing falls under the Bayyinah al-Maal application: claim documents whose surface presentation does not

match their substrate (procedure code disagreeing with attached medical record, billing volume exceeding documented patient encounters) constitute structural-honesty violations that can be caught at the document layer before payment. The dollar impact frame is straightforward: US healthcare spending exceeds $4.5 trillion annually, with $760 billion to $935 billion in estimated annual waste from administrative complexity, fraud, and medical errors. The lives frame is more direct: if software-originated errors contribute to even five percent of the 250,000 annual US medical error deaths, that is 12,500 deaths per year from systems that claimed one thing and did another.

### 23.1.7 Quantitative finance and trading infrastructure

Quantitative hedge funds, prime brokerages, and execution platforms operate complex pipelines where every stage's output feeds the next stage's input: market data ingest, normalization, feature engineering, signal generation, portfolio optimization, order generation, execution, settlement, reconciliation. The framework solves a problem this industry rarely names but consistently loses money to: infrastructure that lies about the state of the world. furqan-lint's D11 (status coverage) checker applied to a quant data pipeline flags every function returning Optional[DataFrame] or Result<PriceVector, DataError> where the downstream consumer assumes data is always present — the exact failure mode where get_latest_prices returns None on a feed timeout and calculate_signal silently computes on None, producing a nonsensical signal that generates a real trade. The additive-only diff applied to a data-layer API catches the case where renaming adjusted_close to adj_close updates the strategy code but not the risk-management consumer, which then silently reads None where it expects a number. Risk-management systems make claims (“maximum drawdown bounded at 5%,” “position sizing never exceeds 2% of NAV per name”) whose substrate is the code implementing them; the four-place pattern (§11) applied to risk limits requires every limit to have a CHANGELOG entry, a README entry, a fixture demonstrating the limit firing, and a pinning test asserting the limit is enforced in current code. A risk limit that exists in policy but has no corresponding test is a finding; a limit changed in code but not in policy is a finding. Reconciliation pipelines that compare internal records to prime broker, exchange, and administrator records are document-layer applications of Bayyinah Integrity Scanner (Arfeen 2026b, 2026c): a trade confirmation PDF with concealed layers or metadata anomalies receives a mushtabih verdict before the reconciliation logic accepts it as ground truth. Bayyinah al-Maal (Arfeen 2026g) applies to the NAV calculation pipeline: a NAV statement where the sum of position values does not equal the stated total is a structural-honesty violation between the document's surface and its computational substrate. The commercial frame is honest about expected value: a strategy with a 0.5 Sharpe ratio running on structurally honest infrastructure outperforms a strategy with a 0.7 Sharpe ratio running on infrastructure that silently drops errors twice a month,

because the error-dropping infrastructure produces outsized losses on exactly the tail events that matter most.

### 23.2 Where the framework does not fit

The discipline of saying no is part of structural honesty. The framework is a poor fit for three classes of project, in each case for the same root reason: the framework's overhead scales with the cost of failure, and where the cost is low, the overhead becomes ceremony.

- Prototypes and hackathon projects. The overhead kills velocity. Ship first, add discipline later if the prototype graduates to a product.
- Solo developer projects with no external consumers. The four-place pattern serves consumers; if there are none, it is documentation for nobody.
- Projects where the cost of a bug is low — internal dashboards, one-off scripts, data-exploration notebooks. The framework's value scales with the cost of failure; below a threshold the discipline costs more than the bugs it prevents.

### 23.3 The highest-leverage deployment path

The framework as a static document is read once and remembered partially. The framework as a tool is enforced on every commit and remembered structurally. The highest-leverage deployment today is to package the framework as a continuous-integration action: a project adds one line to its workflow configuration, the action runs the four-place completeness check, the CHANGELOG-math gate, the version-sync verification, and the additive-only diff, and every pull request receives a structural-honesty audit for free. The framework stops being a document teams read and becomes a tool teams use. This is the operational path that tryfurqan.com instantiates: the framework made executable, applied across three programming languages, running in a browser. The document is the proof. The tool is the product.

### 23.4 The Maturity Model

Organizations adopting the framework progress through five maturity levels. The model is offered as a self-positioning vocabulary; an organization at Level 2 can name what it has and what is missing without negotiating definitions on every conversation. The transition criteria between levels are observable structural properties of the project's substrate, not opinions about the team's culture or intent.

**Figure 10. Bayyinah Audit Framework maturity model**

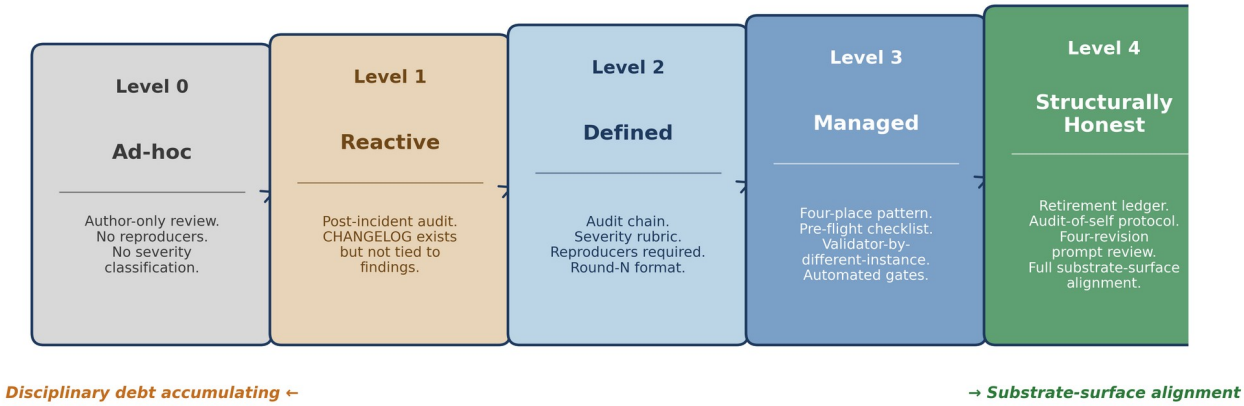



*Figure 10. The Bayyinah Audit Framework maturity model: five levels with observable transition criteria.*

**Level 0 — Ad-hoc.** AI-generated code is reviewed by author only. No reproducer requirement. No severity classification. CHANGELOG entries, if present, do not tie to findings. The most common starting state for AI-assisted projects.

**Level 1 — Reactive.** Audit happens after incidents. CHANGELOG documents fixes after the fact. No automated gates. Findings are surfaced when something breaks in production rather than before commit.

**Level 2 — Defined.** Audit chain established with the severity rubric (§7). Findings have reproducers. Round-N report format used. Limitations are acknowledged but not yet systematically tracked across surfaces.

**Level 3 — Managed.** The four-place documentation pattern (§11) is enforced. The pre-flight checklist (§12.1) is applied before every release. The validator-by-different-instance protocol (§5.2) is in routine use. Automated gates run on continuous integration.

**Level 4 — Structurally Honest.** Full disciplinary stack in operation: every claim has a substrate, every finding has a reproducer, every limitation has both a fixture and a pinning test, the retirement ledger is maintained across versions, the audit-of-self protocol is followed where applicable, and the four-revision prompt review chain (§12.3) governs AI-executed corrective sequences. The empirical signature is a monotonically declining severity slope across audit rounds, of the kind documented in §17.

The transition from Level 3 to Level 4 is empirically the hardest. Level 3 organizations have the discipline but not yet the structural mechanism that prevents discipline from drifting into ceremony. Level 4 is reached when the gates, the retirement ledger, and the validator-by-different-instance protocol together carry enough of the load that an individual contributor's vigilance is no longer the load-bearing reliability mechanism.

## 24. The Accountability Use Case

Every application domain in §23 assumes the framework's user is the operator: an engineering team adopting the framework to audit its own substrate. There is a category of application where this assumption inverts. In domains of structural power asymmetry — software that controls human freedom or compounds human suffering — the operator has no incentive to apply structural-honesty discipline because the structural dishonesty is the business model. Private prison companies profit from longer sentences. Debt buyers profit from collecting debts that may not be valid. Algorithmic lenders profit from opaque pricing. The framework's tools become accountability instruments wielded by advocates against operators, not engineering instruments wielded by operators on themselves. This section names four such domains and articulates the deployment pattern that distinguishes them from the operator-self-audit model that grounds the rest of this paper.

### 24.1 The inverted user model

In the operator-self-audit model documented in §22 (Operational Integration), the user of the framework is the team building the software. Audit findings flow into the next release. Severity classifications drive engineering priorities. The four-place pattern serves the team's own future selves and downstream consumers within the same organizational frame. The accountability use case inverts every part of this. The user is not the team building the software; the user is an advocate — a public defender, legal aid attorney, consumer rights organization, or investigative journalist — working on behalf of the people the software acts upon. Audit findings do not flow into the operator's next release; they flow into court filings, regulatory complaints, and FOIA-driven discovery. Severity classifications do not drive engineering priorities; they drive harm taxonomies admissible as evidence. The four-place pattern serves not the operator's downstream consumers but the legal record establishing that a system claimed one thing and substrate did another. Source code access, where it exists, comes through legal discovery, regulatory examination, or open-records requests, not through the team's own version control. The framework's vocabulary remains the same; its institutional surface inverts.

### 24.2 Sentence calculation and corrections software

Sentence calculation systems used by public corrections agencies and private prison contractors compute release dates from the original sentence, statutory good-time credits, earned-time credits, pre-trial detention credit, consecutive versus concurrent sentence stacking, mandatory minimum overrides, and jurisdiction-specific rules subject to legislative amendment. The computation chain is deep, and each step can silently fail. A sentence calculator that returns ReleaseDate when its substrate may have silently

dropped a credit, failed to apply a legislative change, or miscomputed concurrent-sentence overlap is a status-coverage violation in furqan-lint's D11 sense: the function declares it computed a release date while its substrate quietly produced an incorrect one. The person serving extra months has no way to verify the computation; the displayed release date is treated as ground truth, the substrate is opaque. Bayyinah Integrity Scanner (Arfeen 2026b, 2026c) applied to documents in an inmate's file verifies whether the sentencing order matches the data in the calculation system; a sentencing PDF with concealed layers, metadata inconsistencies, or structural anomalies receives a mushtabih verdict that should trigger investigation before a person's freedom continues to depend on it. The framework's four-place pattern applied to calculation rules requires every rule to have a documented policy citation, a user-facing summary, a test fixture demonstrating the rule's application, and a pinning test asserting the calculation produces the correct result for known inputs. A rule changed by legislation but not updated in the software is a CRITICAL finding, because every day the software runs with the old rule someone is being held illegally. Public defenders and legal aid organizations are the natural users; the deliverable is a court-admissible report, not an engineering ticket.

### 24.3 Risk assessment algorithms

Algorithmic risk assessment tools — COMPAS, LSI-R, and similar instruments deployed in pretrial release decisions, sentencing, and parole — produce scores that determine where a person is housed, what programs they can access, and when they are eligible for release. The systems are typically proprietary, trained on data that may encode racial, economic, or geographic biases, and presented to judges as objective scores when their substrate is a statistical model with documented limitations. The 2016 ProPublica investigation of COMPAS established that the algorithm was approximately twice as likely to falsely flag Black defendants as future criminals compared to white defendants — the false-positive rate for one population structurally different from the false-positive rate for another, but the score presented identically to both. In the framework's vocabulary this is an East-West asymmetry (§9.4) presented at the system surface as if no asymmetry existed. The four-place pattern applied to algorithmic risk assessment would require every documented limitation — demographic disparities in error rates, training-distribution boundaries, and known failure modes — to be visible at every surface where the score is consumed: a fixture demonstrating the disparity, a pinning test asserting the disparity persists in the current model, a CHANGELOG entry naming when the disparity was discovered, and a README bullet visible to every judge using the score. The judge currently sees “high risk”; the framework's discipline would require the judge to see, at the same surface, the model's documented false-positive-rate disparity. Source code access for these systems typically requires litigation discovery or regulatory examination,

which means the framework's natural institutional users are civil-rights organizations and academic researchers rather than the operators themselves.

### 24.4 Debt collection and chain-of-title verification

Debt collection software pursues debts that may be time-barred, already paid, discharged in bankruptcy, or attributable to a different person. Debt buyers acquire portfolios for pennies on the dollar with frequently incomplete or inaccurate provenance data; the collection software loads the portfolio, generates demand letters, and files lawsuits based on the data it received. Bayyinah Integrity Scanner applied to debt documentation verifies the structural integrity of the chain of title: does the original creditor's record match the debt buyer's record, does the account number on the demand letter trace to the original credit agreement, does the stated balance derive from a verifiable original balance through a documented chain of transactions? A demand letter whose stated balance does not trace through that chain receives a mushtabih verdict, providing the debtor's attorney or legal aid representative with a structural basis for challenging the debt. The framework applied to collection software requires every function determining “this debt is valid and collectible” to document its decision logic with the four-place pattern. A statute-of-limitations check is a function that can return Collectible | TimeBarred; if the downstream demand-letter generator consumes the result as Collectible without handling the TimeBarred case, the D11 violation is not abstract — it means a real person receives a collection letter for a debt that cannot legally be collected, and most people pay because they do not know their rights. Credit-reporting systems exhibit a related failure mode: a Federal Trade Commission study found one in four consumers identified errors on their credit reports that might affect their scores. The additive-only diff applied to credit-report data catches the reinsertion failure mode (a disputed item is removed in cycle T, reappears in cycle T+1) as a structural violation rather than a judgment call: the negative item was removed, then reappeared, which violates the additive-only contract on dispute resolutions.

### 24.5 Predatory lending and discriminatory pricing

Subprime auto loans, payday lending, rent-to-own contracts, and high-interest credit cards use algorithmic underwriting that determines interest rates, fees, and terms. The pricing logic is opaque to the borrower; a borrower offered a 24% APR auto loan does not know whether the algorithm would have offered a different rate to a borrower with identical creditworthiness but a different ZIP code. The structural-honesty diagnostic from Al-Mutaffifin (Arfeen 2026h) applies in its purest form: a system that measures differently depending on who is being measured. The framework's check is whether the pricing function produces the same output for the same risk inputs regardless of protected characteristics; a function where price(risk=X, zip=A) differs from price(risk=X, zip=B)

when ZIP code is not a legitimate risk factor is a structural-honesty violation. Bayyinah al-Maal (Arfeen 2026g) applied to loan documents verifies that the disclosed APR, the payment schedule, and the total cost of credit are mathematically consistent; a loan document where the disclosed APR and the actual payment stream produce different total costs is a document-level surface-substrate gap, exactly the discrepancy documented in Consumer Financial Protection Bureau enforcement actions against multiple lenders. The natural institutional users are fair-housing organizations, the CFPB, state attorneys general, and academic researchers obtaining algorithm documentation through regulatory examination.

### 24.6 Deployment pattern: framework as evidence standard

The accountability use case changes three deployment properties without changing the framework's diagnostic vocabulary. First, the user changes from operator to advocate: legal aid organizations, public defenders, civil-rights groups, consumer-rights coalitions, and investigative journalists. Second, the output format changes from “fix the bug in the next release” to a documented finding with a reproducer, formatted to be admissible as evidence in court, regulatory complaint, or public reporting. The four-place pattern becomes an evidence standard: a system claims X, the substrate does Y, here is the reproducer, here is the citation. Third, the severity rubric becomes a harm taxonomy. A CRITICAL finding in a sentence calculation system means a person is being unlawfully detained. A HIGH finding in a credit scoring system means a person is paying more than their risk profile justifies. The translation from engineering severity to civic harm is direct because the framework's severity rubric is grounded in impact, not in operator inconvenience.

A calibration is required, in the same shape as the calibration in §19. The framework cannot fix the power asymmetry that allows these systems to operate without accountability; private prison companies, debt collectors, and predatory lenders will continue to operate within the institutional structures that protect them regardless of any audit framework. What the framework can do is make the structural dishonesty visible, reproducible, and legally actionable. A finding with a reproducer is evidence; evidence changes outcomes within institutions designed to respond to evidence. The discipline is the same as for operator-side audit. The substrate is the same. The stakes are human freedom and dignity rather than enterprise revenue. Detailed deployment guidance for the legal aid community, civil-rights organizations, and regulatory bodies is the subject of a forthcoming companion brief; this section establishes only that the framework's diagnostic categories apply to power-asymmetric domains and that the deployment pattern differs from the operator-self-audit model that grounds the rest of this paper.

## 25. Code and Data Availability

The three sibling repositories that grounded the framework's empirical validation are publicly hosted under the BayyinahEnterprise organization on GitHub. Reviewers wishing to verify the claims in §17 (Empirical Trajectory and Retirement Ledger) and §18 (Reference Examples from the Audit Chain) can clone each repository at the version cited and reproduce the documented behavior using the project's own continuous-integration gate suite.

| Repository | URL | Release |
|---|---|---|
| Bayyinah Integrity Scanner | https://github.com/BayyinahEnterprise/Bayyinah-Integrity-Scanner | **v1.2.3** |
| Furqan programming language | https://github.com/BayyinahEnterprise/furqan-programming-language | **v0.11.1** |
| furqan-lint | https://github.com/BayyinahEnterprise/furqan-lint | **v0.8.5** |
| furqan-lint (PyPI) | https://pypi.org/project/furqan-lint/ | **v0.8.5** |

For each repository, the verification path is identical: clone at HEAD or at a specific release tag; install development dependencies via the project's pyproject configuration or equivalent; run the gate suite from a clean checkout. CHANGELOG entries name the audit round in which each finding was surfaced and the version in which it was retired. The four-place documentation pattern (§11) ensures any documented limitation is reachable from CHANGELOG, README, fixture file, and pinning test together; reviewers may begin from any of the four surfaces and arrive at the same substrate.

All three repositories are licensed for review and audit purposes. Specific re-use terms vary by repository and are stated in each project's LICENSE file. This document itself is archived at Zenodo under the DOI listed on the title page.

## 26. References

References are listed in two groups, alphabetized within each group. The first group lists the BayyinahEnterprise companion publications cited in §20.1, §23.1, and §24, all open-access on Zenodo. The second group lists the external primary sources cited in §19 (Counterfactual Application).

### 26.1 BayyinahEnterprise Companion Publications

Arfeen, B. S. (2026a). Detecting Performed Alignment in Artificial Systems — The Munafiq Protocol (v2.1). Zenodo. https://doi.org/10.5281/zenodo.19700420

Arfeen, B. S. (2026b). Bayyinah: Detecting Concealed Adversarial Content in Digital Documents — A White Paper Applying the Munafiq Protocol to the Input Layer (v.1). Zenodo. https://doi.org/10.5281/zenodo.19745154

Arfeen, B. S. (2026c). Bayyinah as Input-Layer Defense in Artificial-System Safety Pipelines (v1.1). Zenodo. https://doi.org/10.5281/zenodo.19802455

Arfeen, B. S. (2026d). Al-Khalifa: A Furqan-Based Super Agent Architecture for Structurally Honest Autonomous Project Stewardship (v2). Zenodo. https://doi.org/10.5281/zenodo.19776577

Arfeen, B. S. (2026e). Bilal: An Honest-Autonomous Large Language Model Architecture with Structural Truth Verification, Calibrated Generation, and Purpose-Hierarchy Training Objectives Derived from Quranic Computational Architecture (v2). Zenodo. https://doi.org/10.5281/zenodo.19776576

Arfeen, B. S. (2026f). The Fatiha Construct: A Seven-Step Recursive Session Protocol for Human-AI Collaborative Development Derived from Surah al-Fatiha (v.1). Zenodo. https://doi.org/10.5281/zenodo.19746539

Arfeen, B. S. (2026g). Bayyinah al-Maal: Structural Honesty Verification for Financial Filings Using the Munafiq Protocol (v1.0). Zenodo. https://doi.org/10.5281/zenodo.19875931

Arfeen, B. S. (2026h). Al-Mutaffifin: Structural Integrity Verification for Financial Governance Systems (v1.2). Zenodo. https://doi.org/10.5281/zenodo.19894724

Arfeen, B. S., & Ashraf, F. (2026a). Bayyinah al-Khabir: A Theoretical Framework for Information-Layer Integrity Scanning Across National Broadcast Sources Using Performed-Alignment Diagnostics (v1.0). Zenodo. https://doi.org/10.5281/zenodo.19746298

Arfeen, B. S., & Ashraf, F. (2026b). Cross-Text Computational Linguistics: Structural Properties of the Quran, Hadith Collections, and the Hebrew Bible Under TF-IDF Cosine Analysis (v1). Zenodo. https://doi.org/10.5281/zenodo.19961267

Arfeen, B. S., Claude (Opus 4.6, Anthropic), & Grok (xAI). (2026). Structured Revelation as Prompt Architecture (v2). Zenodo. https://doi.org/10.5281/zenodo.19744163

Ashraf, F. R., & Arfeen, B. S. (2026). Furqan: A Programming Language with Structural Honesty, Calibrated Optimization, and Surface-Depth Type Verification Derived from Quranic Computational Architecture (v2). Zenodo. https://doi.org/10.5281/zenodo.19776584

### 26.2 External Primary Sources

APAS (Wirtschaftsprüferaufsichtsstelle). (2021). Investigation findings on EY's Wirecard audits 2016–2018. Reported in German parliamentary review materials and subsequent public disclosures (2024).

Barr, M. (2013). Bookout v. Toyota Motor Corporation — Expert witness reports on the Camry L electronic throttle control source code. District Court of Oklahoma County, Oklahoma.

House Transportation and Infrastructure Committee. (2020). Final Committee report on the design, development, and certification of the Boeing 737 MAX. U.S. House of Representatives.

Leveson, N. G., & Turner, C. S. (1993). An investigation of the Therac-25 accidents. IEEE Computer, 26(7), 18–41.

NERC (North American Electric Reliability Council). (2004). Final report on the August 14, 2003 blackout in the United States and Canada: Causes and recommendations.

NHTSA & NASA. (2011). Technical assessment of Toyota electronic throttle control (ETC) systems. U.S. Department of Transportation report. https://www.nhtsa.gov/sites/nhtsa.gov/files/nhtsa-ua_report.pdf

NTSB (National Transportation Safety Board). (2019). Safety recommendation report: Assumptions used in the safety assessment process and the effects of multiple alerts and indications on pilot performance (ASR-19-01). National Transportation Safety Board.

## 27. Acknowledgements

I thank Fraz Ashraf for the engineering approach that grounds this framework. The substation-tradition discipline that frames every claim, every finding, and every retirement traces to his contribution; this framework is the hybridization of his engineering approach with my work on Bayyinah Integrity Scanner, the Furqan programming language, and furqan-lint. The framework documented here is sole-authored, but the discipline behind it is shared.

Audit drafts and code review were assisted by Claude Sonnet 4.6 and Claude Opus 4.6 / 4.7 (Anthropic) and Perplexity Computer (Comet); literature engagement was assisted by Grok (xAI). All findings, verdicts, and final framework decisions are the responsibility of the author.

This document is licensed under Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0). Commercial use requires a separate license:

bayyinahenterprises@tuta.com. Citations of this work should reference the Zenodo DOI assigned upon upload.

---

*Version 3.0 · May 3, 2026 · CC BY-NC 4.0 · Bilal Syed Arfeen*